Highlights

Structurally-Calibrated Functional Attribution: A Methodology for Process Supervision Weighting in Reflective Reasoning

Anonymous Author(s)

- SC-FMA calibrates interventional utility into supervision weights.
- A convex SCU objective protects structure, redundancy, and bottlenecks.
- SC-FMA improves step-ranking correlation over raw CIU and six baselines.
- Ridge, QP, and Projection variants trade speed for ranking accuracy.

Structurally-Calibrated Functional Attribution: A Methodology for Process Supervision Weighting in Reflective Reasoning

Anonymous Author(s)[a]

[a]*Anonymous Institution, Anonymous, Anonymous*

ARTICLE INFO

Keywords:
structural calibration
functional attribution
process supervision
convex optimization
reflective cognition
step importance ranking

ABSTRACT

Knowledge-intensive reasoning systems increasingly rely on reflective steps such as self-evaluation, error diagnosis, and plan revision to organize verification workflows. While local interventional utility identifies which reflective steps correlate with successful outcomes, prior work shows that local utility is weakly aligned with structural necessity—many locally useful steps are topologically redundant. This paper presents Structurally-Calibrated Functional Attribution (SC-FMA), a methodology for auditable verification-step weighting in knowledge-intensive reasoning systems. SC-FMA converts interventional utility estimates into calibrated supervision weights through constrained convex optimization. The core contribution is the Structurally-Calibrated Utility (SCU) objective, which jointly optimizes fidelity to local utility signals, consistency with graph-level structural necessity, redundancy reduction, and bottleneck protection. The objective is strictly convex, guaranteeing a unique global solution, and provides formal monotonicity and variance-reduction guarantees. We implement three calibration variants—Ridge, Quadratic Program (QP), and Topology Projection—spanning a simplicity-power spectrum. On step importance ranking with synthetic and real annotated benchmarks, SC-FMA QP achieves a mean Spearman ρ of 0.608 with oracle step labels, outperforming raw CIU (0.483), gradient attribution (0.491), Monte Carlo Shapley (0.524), and information-theoretic baselines (0.310). An ablation study confirms that each structural constraint independently contributes to ranking quality: structure alignment (+0.057), redundancy penalty (+0.022), and bottleneck protection (+0.013). SC-FMA provides a principled bridge from intervention-based attribution to process-supervision-oriented analysis, and its convex formulation supports theoretical analysis and auditable implementation.

1. Introduction

Knowledge-intensive reasoning systems require reliable verification workflows: intermediate checks must be useful, but they must also occupy the right structural position in an inference chain. In expert systems, knowledge graphs, and reflective language-agent pipelines, a verification step may be locally correlated with success while still being redundant because an alternative rule, entity path, or neighboring reflection supplies the same support. Conversely, a rare bottleneck check may carry little local utility signal but still gate downstream inference. This makes process supervision a KBS problem as well as a scoring problem: supervision weights should reflect local utility, structural dependency, redundancy, and bottleneck protection.

The dominant approach treats any reflective operation that correlates with improved final-answer quality as a candidate supervision weight. This intuition is attractive: a model that checks, revises, or critiques and gets a better answer must have produced useful intermediate steps. However, prior diagnostic work reveals a structural tension [30, 56]: up to half of all locally useful reflective steps exhibit zero measured structural necessity when the reasoning graph is perturbed, and the linear correlation between local utility and graph-level necessity is weak (Pearson < 0.10). Compensatory redistribution is limited, and structural influence is concentrated rather than distributed. Together, these findings imply that naive utility-based supervision would over-weight fluent but redundant reflection while under-weighting rare structurally critical operations.

This paper proposes a methodological solution. We present **Structurally-Calibrated Functional Attribution (SC-FMA)**, a methodology that transforms raw interventional utility estimates into calibrated, topologically-consistent supervision weights through constrained convex optimization. Rather than treating structural diagnostics as a separate analysis layer, SC-FMA embeds them directly into the weight computation.

ORCID(s):

The core of SC-FMA is the **Structurally-Calibrated Utility (SCU) objective**:

$$L(\mathbf{w}) = \alpha\|\mathbf{w} - \tilde{\mathbf{c}}\|_2^2 + \beta\|\mathbf{w} - \tilde{\mathbf{n}}\|_2^2 + \gamma\, \mathbf{w}^\top \mathbf{R}\mathbf{w} - \delta \sum_i b_i \log(w_i) \tag{1}$$

subject to $w_i \geq 0$ and $\sum_i w_i = 1$, where $\tilde{\mathbf{c}}$ is normalized CIU, $\tilde{\mathbf{n}}$ is structural necessity, $\mathbf{R}$ encodes pairwise redundancy, and b_i identifies bottleneck nodes. The four terms enforce: (1) fidelity to the local utility ranking, (2) consistency with graph-level necessity, (3) a penalty for assigning dissimilar weights to redundant pairs, and (4) a log-barrier preventing bottleneck nodes from receiving near-zero weight.

The SCU objective is strictly convex, guaranteeing a unique global minimizer when any regularization weight is positive. It provides four formal guarantees: **G1 (convexity and uniqueness)** of the solution; **G2 (monotonicity)**—for non-redundant step pairs, the calibrated weights preserve the joint CIU and necessity ordering; **G4 (variance reduction)**—SC-FMA weights have provably lower variance than raw CIU weights for any positive structural regularization; and **G6 (bottleneck protection)**—identified bottleneck nodes never receive zero weight regardless of their CIU signal strength.

We implement three calibration variants spanning a simplicity-power spectrum. SC-FMA Ridge uses a closed-form softmax over a linear combination of CIU and necessity ($O(k)$ per trace). SC-FMA QP solves the full constrained quadratic program with redundancy and bottleneck constraints ($O(k^3)$ but converges in under 10ms for typical reasoning traces of 3–8 steps). SC-FMA Projection applies a fast topology-constrained projection using elementwise operations.

We evaluate SC-FMA on step importance ranking—an evaluation task that measures how well predicted supervision weights correlate with oracle step-level correctness labels. We compare against six baseline families: (A) gradient attribution (Gradient×Input, attention rollout), (C) Monte Carlo Shapley value, (D) information-theoretic (token surprisal, step entropy), (E) heuristic (random, span length, relative position), and (F) oracle labels (ground-truth ceiling). All experiments are reproducible with deterministic seeds and documented hyperparameters.

Our primary empirical findings are:

1. SC-FMA QP achieves a mean Spearman ρ of 0.608 with oracle step labels, representing a +0.125 improvement over raw CIU (0.483, $p < 0.01$, Wilcoxon signed-rank).
2. Each structural constraint term independently contributes to ranking quality in ablation analysis: removing structure alignment costs 0.057, removing redundancy penalty costs 0.022, and removing bottleneck protection costs 0.013.
3. SC-FMA significantly outperforms gradient attribution (0.491, $p = 0.003$), Monte Carlo Shapley (0.524, $p = 0.018$), and token surprisal (0.310, $p < 0.001$) baselines.
4. The Friedman omnibus test confirms significant differences across all methods ($\chi^2 = 142.3$, $p < 0.001$).

The contributions of this paper are: (1) the SC-FMA methodology that transforms interventional utility into structurally-calibrated supervision weights; (2) the SCU objective with formal convexity, monotonicity, variance-reduction, and bottleneck-protection guarantees; (3) a step importance ranking evaluation framework against six baseline families; (4) empirical evidence that structural calibration improves ranking quality over raw utility and standard baselines; and (5) an implemented, open-source algorithm with configurable variants.

2. Notation

Table 1 summarizes the mathematical notation used throughout the paper. We adopt the convention that vectors are denoted in bold lowercase ($\mathbf{w}$, $\mathbf{c}$, $\mathbf{n}$), matrices in bold uppercase ($\mathbf{R}$), and scalars in standard typeface (α, β, k). Sets are denoted in calligraphic typeface ($\mathcal{V}$, $\mathcal{E}$, $\mathcal{W}$), and graph objects in script style ($\mathcal{G}$). All vectors are column vectors unless otherwise noted; $\|\cdot\|_2$ denotes the Euclidean norm, and $\odot$ denotes elementwise (Hadamard) multiplication.

3. Related Work

SC-FMA bridges five research areas: process reward models and step-level supervision, attribution and interpretability for reasoning, graph-based analysis of reasoning, convex optimization in machine learning, and knowledge-based/cognitive systems. This section positions SC-FMA within each thread and identifies the gaps it addresses.

3.1. Process Reward Models and Step-Level Supervision

Process Reward Models assign scalar value estimates to intermediate reasoning steps. Lightman et al. [30] established the standard PRM training paradigm: human annotators label each reasoning step as correct or incorrect, and a model is trained to predict these labels, enabling best-of-N search and reinforcement learning from process feedback. Uesato et al. [53] compared process-based and outcome-based feedback for math word problems, demonstrating that process supervision provides denser training signals than final-answer rewards alone. Cobbe et al. [11] introduced training verifiers that score intermediate completions, establishing verifier-guided search as a complementary mechanism to supervised fine-tuning.

Recent work reduces the human annotation bottleneck through automated verification. Math-Shepherd [56] automates process annotation by measuring whether masking a step increases the probability of an incorrect completion, producing pseudo-labels without human judges. Luo et al. [33] extend this direction with Monte Carlo Tree Search over reasoning paths, using completion consistency as an automated signal. ProcessBench [63] provides a standardized evaluation for step-level error identification, exposing the brittleness of process supervision under distribution shift. Zheng et al. [63] show that even strong PRMs fail to detect over 20% of step-level errors in out-of-distribution math problems.

A common assumption across all these methods is that step-level correctness—whether human-annotated or automatically inferred—constitutes a valid supervision target. However, diagnostic evidence from reflective reasoning traces reveals that local utility (whether a step correlates with correct outcomes under masking) and structural necessity (whether the step is topologically irreplaceable in the reasoning graph) are weakly aligned, with Pearson correlations below 0.10 and up to half of locally useful steps exhibiting zero measured structural necessity. This implies that treating step correctness as direct supervision risks over-weighting fluent but structurally redundant reflection while under-weighting rare, structurally critical operations.

Unlike PRM-based approaches that treat step correctness as a supervision endpoint, SC-FMA operates as a *pre-calibration module*. It accepts raw interventional utility estimates (including PRM scores, CIU signals, or verifier outputs) and produces topologically-calibrated weights through constrained convex optimization before those weights are used as supervision targets. SC-FMA is therefore a complement to, rather than a replacement for, existing PRM pipelines. It addresses the structural redundancy gap that PRMs do not model.

3.2. Attribution and Interpretability for Reasoning

Attribution methods quantify how input components contribute to model predictions. At the token level, LIME [46] constructs local linear surrogates around individual predictions, identifying which input tokens drive a specific output. Integrated Gradients [51] accumulates gradients along a path from a baseline input, satisfying axioms of sensitivity and implementation invariance. SHAP [32] unifies additive feature attribution under Shapley values from cooperative game theory, providing theoretically grounded importance scores. These methods have been widely adopted in NLP for tasks such as sentiment classification, natural language inference, and question answering [24, 58].

Several limitations emerge when these methods are applied to reasoning traces. First, they operate at the token level, while reflective reasoning operations—self-evaluation, error diagnosis, plan revision—span multi-token phrases whose semantic function cannot be decomposed into independent token contributions. Second, they measure input-output sensitivity rather than structural role: a locally sensitive token may be redundant if its function can be fulfilled by neighboring spans. Third, they lack topological awareness—the position of an explanation unit within the reasoning flow (early verification vs. late correction, bottleneck vs. leaf) is not modeled.

Step-level attribution methods partially address the granularity concern. ERASER [13] evaluates rationalized NLP models by measuring whether extracted rationales are both plausible and faithful. Attention flow [1] tracks information propagation through transformer layers, while self-attention attribution [21] identifies which token interactions drive layer-wise representations. However, these methods explain *what* contributes to a prediction, not *how* the structural relationships among reasoning steps determine their collective necessity. They treat steps as independently attributable units without modeling the dependencies between them (redundancy, compensation, bottleneck formation).

Unlike token-level attribution methods that explain individual predictions, SC-FMA calibrates the *structural relationships* among steps within a reasoning trace. Unlike step-level rationalization methods that identify important spans, SC-FMA asks how those spans' importance interacts with the topology of the reasoning graph. The shift is from "which step is locally useful?" (attribution) to "how should supervision weights respect step-step structural dependencies?" (structural calibration). SC-FMA does not discard attribution; it uses attribution (CIU) as input and

applies structural constraints to resolve the gap between locally high-attribution steps and their topologically redundant role.

3.3. Graph-Based Analysis of Reasoning

Explicit graph representations of reasoning have gained prominence as alternatives to linear chain-of-thought. Tree of Thoughts (ToT) [60] organizes reasoning as a search tree, where each node is an intermediate thought and branching enables exploration of multiple reasoning paths via BFS or DFS with pruning. Graph of Thoughts (GoT) [6] generalizes this to arbitrary directed graphs, supporting operations such as aggregation, refinement, and backtracking across concurrent reasoning branches. These methods demonstrate that non-linear reasoning structures improve performance on tasks requiring exploration, planning, and constraint satisfaction. However, they use graph structure as a *generation scaffold*—a mechanism for producing better reasoning traces—rather than as a *diagnostic lens* for understanding how existing reasoning steps are organized.

Graph explanation methods study how structural interventions reveal component importance. GNNExplainer [61] identifies the minimal subgraph and feature subset that best explain a graph neural network's prediction, using mutual information maximization. PGM-Explainer [54] extends this with probabilistic graphical model approximations. These methods target GNN decision boundaries; SC-FMA transposes the logic to *reflective reasoning traces*, where nodes are explicit metacognitive operations and edges encode temporal and topical dependencies.

Network science provides the theoretical foundation for structural diagnostics. Albert et al. [2] demonstrated that complex networks exhibit fundamentally different robustness profiles under random failures (where most nodes are redundant) versus targeted attacks (where hub removal causes catastrophic fragmentation). Motter and Lai [37] showed that cascading failures propagate along load-bearing paths, creating indirect vulnerabilities absent from direct node importance measures. Newman [39] formalized the distinction between local centrality measures (degree, betweenness) and global vulnerability metrics (percolation threshold, giant component fraction). These network-science insights directly inform the PRUNE (direct node removal), CASCADE (downstream propagation), and BYPASS (alternative path substitutability) interventions used in SC-FMA's structural diagnostics.

Unlike GoT and ToT, which use graph structure to *generate* better reasoning traces, SC-FMA uses graph structure to *calibrate* supervision weights over already-recorded traces. Unlike GNNExplainer, which explains GNN predictions, SC-FMA explains step-step functional relationships within a reasoning graph. Unlike network science applied at the infrastructure level (Internet, power grids), SC-FMA applies the same structural principles—redundancy, bottleneck protection, cascade sensitivity—to the organization of reflective cognition in language-agent traces.

3.4. Convex Optimization in Machine Learning

Convex optimization provides the mathematical machinery for SC-FMA's weight calibration. Quadratic programming (QP) has been a workhorse in machine learning since Cortes and Vapnik [12] formulated the support vector machine as a convex QP with linear constraints. Markowitz [35] pioneered mean-variance portfolio optimization, where the objective balances expected return against risk through a convex combination of linear and quadratic terms—a structural precedent for SC-FMA's multi-term SCU objective. Boyd and Vandenberghe [8] provide the canonical treatment of convex optimization, establishing conditions for unique global solutions (strict convexity), duality theory, and interior-point methods.

The log-barrier technique, central to SC-FMA's bottleneck protection term, originates from interior-point methods for constrained optimization [38]. By adding $-\sum_i b_i \log(w_i)$ to the objective, the optimizer is repelled from the boundary $w_i = 0$, guaranteeing that identified bottleneck nodes receive strictly positive weight regardless of their raw CIU signal strength. Nocedal and Wright [41] provide convergence guarantees for sequential quadratic programming (SQP) methods, which SC-FMA's QP variant adopts through the SLSQP solver [26].

Sparse optimization and structured regularization provide additional conceptual precedent. The LASSO [52] introduces ℓ_1 regularization to promote sparsity in linear regression coefficients, demonstrating that structural priors (sparsity) can be encoded as convex penalty terms. Group LASSO [62] extends this to grouped variable selection, where entire groups of features are jointly selected or discarded—analogous to how SC-FMA's redundancy penalty encourages similar weights for topically-linked reflective steps. Projection onto the probability simplex [57] provides the theoretically-grounded mechanism for enforcing the weight normalization constraint $\sum_i w_i = 1$ in SC-FMA Projection.

Unlike standard QP formulations that optimize for a single objective (margin maximization, portfolio risk), SC-FMA's SCU objective is a *multi-term convex combination* where four structurally distinct terms compete under a single

simplex constraint: CIU fidelity, structural necessity consistency, redundancy reduction, and bottleneck protection. Unlike LASSO-style sparsity, which discards features, SC-FMA's structural constraints *redistribute* weight from redundant nodes toward structurally critical ones while preserving a dense weight vector. The formal guarantees—strict convexity, uniqueness, monotonicity, variance reduction, and bottleneck protection—are obtained by proving properties of the composite objective rather than by invoking off-the-shelf optimizer properties.

3.5. Knowledge-Based and Cognitive Systems

3.5.1. Classical Knowledge-Based and Cognitive Systems

The study of reflective reasoning operations has a long precursor in cognitive architectures and knowledge-based systems. SOAR [27] embeds metacognitive monitoring as a separable architectural concern: when routine procedural knowledge reaches an impasse, the architecture triggers deliberate subgoal reasoning in a distinct problem space, explicitly separating automatic execution from reflective problem-solving. ACT-R [3] evaluates production-rule utility through explicit cost-benefit signals, enabling the architecture to monitor and adjust its own processing preferences based on reinforcement learning over declarative and procedural memory. These architectures treat metacognition as a built-in control loop with access to internal state representations.

Expert systems provide an earlier precedent for step-level necessity analysis. MYCIN [9] traced rule-firing chains backward from clinical conclusions to supporting evidence, making individual inference steps transparent and interrogable to clinicians. The explanation facility encoded the dependency structure of the knowledge base, distinguishing necessary intermediate conclusions from corroborating ones. DENDRAL [31] similarly traced its chemical structure elucidation reasoning, exposing which spectrographic constraints contributed to which substructure hypotheses. SC-FMA inherits this concern with auditable inference-step necessity, but it studies observable language-agent traces rather than hand-authored production rules or explicit expert-system dependency graphs.

Formal KBS languages occupy an adjacent but non-baseline role in this paper. OWL 2 provides formally defined ontology semantics for classes, properties, and individuals [22], while answer set programming supports declarative modeling and solving over logic-program constraints [18]. SC-FMA does not implement or compete with these systems. Instead, they indicate the kind of rule, type, or ontology-derived dependency metadata that a future KBS deployment could pass into SC-FMA's graph layer.

3.5.2. Knowledge Graph Reasoning and Graph-Structured LLM Reasoning

Modern knowledge graph reasoning treats paths, rules, and entity-relation structures as inspectable carriers of evidential weight. Path-ranking algorithms [28], differentiable rule learners [59], and rule mining over incomplete ontological knowledge bases [16] formalize the intuition that not every intermediate inference step is equally indispensable. Recent work on LLMs and knowledge graphs further emphasizes the need to coordinate neural reasoning with explicit knowledge structure [25, 42]. Unlike Pan et al.'s broad LLM–KG roadmap, which organizes KG-enhanced LLMs, LLM-augmented KGs, and synergized LLM–KG systems [42], SC-FMA does not propose a general unification architecture; it treats graph structure as a post hoc constraint on supervision weights for already-recorded traces. Direct evaluation of chain-of-thought over multi-hop knowledge graphs shows that reasoning traces can be assessed against graph-grounded intermediate dependencies, not only final answers [40].

This literature also clarifies the distinction between graph structure as a *generation scaffold* and graph structure as a *diagnostic lens*. Tree of Thoughts and Graph of Thoughts use branching or graph-structured reasoning to generate better candidate solutions [6, 60]. SC-FMA instead takes already-recorded reflective traces and asks how their graph-derived structural relationships should constrain verification-step weights. In this sense, SC-FMA does not use knowledge graphs to produce reasoning; it uses graph structure to diagnose redundancy, bottlenecks, and structural necessity in observable reasoning traces.

3.5.3. Neuro-Symbolic and Knowledge-Enhanced Language Models

Neuro-symbolic reasoning systems provide a third line of relevant precedent. End-to-end differentiable proving [47], explanatory rule learning [15], and neural probabilistic logic programming [34] integrate neural scoring with symbolic proof or rule structure. These systems naturally separate local step plausibility from proof-structure necessity: a locally plausible intermediate lemma may be redundant if an alternative proof path exists, while a small unification operation may be structurally critical.

Knowledge-enhanced pre-trained language models and LLM-KG frameworks extend this concern to modern language-model systems by injecting, retrieving, or aligning explicit knowledge during pre-training, prompting, or

reasoning [23, 42]. Hu et al. survey knowledge-enhanced pre-trained language models as systems that incorporate linguistic, textual, knowledge-graph, and rule knowledge into PLMs across understanding and generation settings [23]. SC-FMA is complementary to these approaches. It does not train a knowledge-enhanced model, construct a new knowledge graph, or certify formal reasoning. Instead, it supplies a calibration layer that can receive local utility signals from neural traces and structural constraints from an observed reasoning graph. If future KBS deployments provide ontology-aware edges or rule dependencies, the same SCU objective could use those structures as inputs; in the current paper, the graph remains an operational approximation over surface traces.

3.6. Positioning Summary

Table 2 positions SC-FMA relative to prior work along two axes: the level of analysis (local/individual step vs. structural/relational) and the primary function (diagnostic/explanatory vs. interventional/prescriptive). SC-FMA occupies the "structural + interventional" quadrant, unique among the compared methods.

The diagonal structure of this table reveals a gap in the literature: most methods are either local-diagnostic (explaining what individual steps contribute) or local-interventional (scoring individual steps for supervision), while structural methods have primarily been diagnostic (explaining graph neural networks, analyzing network robustness). SC-FMA fills the "structural + interventional" quadrant by applying constrained convex optimization to calibrate supervision weights against graph-level structural diagnostics. This positioning makes explicit what SC-FMA contributes and what it does not: it provides a methodology for structural calibration rather than a new attribution method, a new PRM, or a new reasoning architecture.

Causal inference terminology clarifies what SC-FMA does *not* claim [43, 48]: it does not estimate Rubin-style causal effects or recover Pearl-style do-calculus semantics. The intervention notation is operational shorthand for documented trace manipulation followed by deterministic evaluation.

3.7. Reproducibility and Data Governance

The methodology inherits reproducibility disciplines from the broader machine learning community. Reproducibility checklists [44], variance-aware experimental reporting [7, 14], and benchmark governance frameworks [29, 45] emphasize that reported results depend on documented data provenance, fixed random seeds, frozen evaluation protocols, and explicit exclusion criteria. SC-FMA adopts these disciplines at the trace level: all 200 synthetic traces are generated with a documented seed (42) and probabilistic model (Section 5.1.3); CIU estimation uses a fixed MASK intervention with deterministic ℓ_2 normalization; graph construction is governed by explicit TF-IDF parameters ($\tau_{\text{tfidf}} = 0.35$, $w_{\text{topical}} = 0.75$); and all hyperparameters are tuned on a validation set that is never intersected with the test set.

Dataset documentation practices [5, 17, 36] further inform the paper's claim boundaries. The synthetic benchmark is fully documented: its generative distributions (CIU as $\text{Beta}(2, 5)$, necessity as zero-inflated $\text{Beta}(3, 2)$, redundancy as block-diagonal structure) are described in Section 5.1.3 and reproducible from the archived generation script. The current positive real-data evidence uses PRM800K process-supervision annotations under a frozen hash split, with explicit provenance and overlap audits. GSM8K and HotpotQA routes are retained only as failed or blocked replay/filtering provenance; they are not used as current validation evidence.

4. Methodology

SC-FMA builds on two diagnostic procedures—Conditional Interventional Utility (CIU) estimation and structural necessity measurement—and adds a novel constrained optimization layer that resolves the tension between local utility signals and global topological constraints. This section provides the complete technical specification for each component.

4.1. CIU Estimation

Given a reasoning trace with k reflective steps identified by a taxonomy parser, CIU quantifies the local functional contribution of each step through structure-preserving intervention.

4.1.1. Intervention Types

Five deterministic intervention operators are defined, each producing a modified trace where step i is altered while the remaining $k - 1$ steps are preserved. All interventions respect the structure-preserving constraint: token count, positional layout, and autoregressive ordering of untouched spans remain identical to the original.

- **MASK.** Replace every token in step i with a reserved sentinel `[MASK]`, preserving the exact token count. This is the primary intervention: it removes semantic content while maintaining positional structure, making it ideal for span-level ablation in free-text reasoning traces.
- **DELETE.** Remove step i entirely and concatenate the preceding and succeeding steps. This perturbs positional structure but is the cleanest test of whether a step can be fully removed without breaking the trace. Applicable only when adjacent steps are semantically independent (no coreference bridging across the deletion point).
- **SHUFFLE.** Randomly permute the token order within step i, destroying sequential information while preserving the bag-of-words. Useful for testing whether step-level utility derives from word choice or word order (e.g., distinguishing formulaic self-evaluation from substantive revision).
- **TRUNCATE.** Retain only the first $\lfloor \ell/2 \rfloor$ tokens of step i, where ℓ is the original token count. Tests whether the step's utility is concentrated in its first half (e.g., a correction statement followed by redundant elaboration).
- **CONTRADICT.** Replace the step with a negation of its core claim, generated via a fixed template (e.g., "The answer is correct" → "The answer is incorrect"). This is the strongest perturbation: it actively introduces misleading information. Used sparingly to detect steps whose absence (MASK) would not reveal their corrective function.

For each trace, CIU is computed using MASK as the default intervention, with SHUFFLE and TRUNCATE serving as robustness checks. DELETE and CONTRADICT are restricted to traces where positional structure or contradiction semantics are interpretable (approximately 60% and 15% of traces, respectively). The raw CIU value is $c_i = U(Y_{\text{orig}}) - U(Y_{\text{int}_i})$, where $U(\cdot)$ is task-level binary correctness (1 for correct, 0 for incorrect). When $U(Y_{\text{int}_i}) > U(Y_{\text{orig}})$ (the intervention *improves* correctness, producing negative CIU), we clamp $c_i \leftarrow 0$, treating the step as non-contributory rather than actively harmful—negative CIU is a separate diagnostic signal reported in failure audits but excluded from the weight calibration pipeline to avoid rewarding detrimental steps.

4.1.2. Functional Validity of Interventions

A functional validity check over the synthetic benchmark confirms that the MASK intervention produces informative and reliable CIU signals. Across all 1,027 reflective steps, the intervention produces a non-zero outcome change in 71.3% of cases (CIU $\neq 0$), indicating that a majority of reflective steps have measurable task-level impact. The distribution of CIU values is right-skewed (mean 0.128, median 0.075, 95th percentile 0.481), consistent with the generative $\text{Beta}(2, 5)$ prior. The alignment accuracy between intervention direction and expected outcome change is 92.1% for MASK (the remaining 7.9% correspond to traces where masking improves correctness, indicating harmful reflection), compared to 54.3% for DELETE and 41.7% for CONTRADICT. This directional reliability supports MASK as the primary CIU estimation intervention, with the more destructive modes reserved for robustness checks and error analysis.

4.1.3. Normalization

We adopt ℓ_2 normalization for CIU and necessity vectors: $\tilde{\mathbf{c}} = \mathbf{c}/\|\mathbf{c}\|_2$, $\tilde{\mathbf{n}} = \mathbf{n}/\|\mathbf{n}\|_2$. This choice is motivated by three considerations. First, the SCU objective uses squared Euclidean distance, which pairs naturally with ℓ_2-normalized targets—minimizing $\|\mathbf{w} - \tilde{\mathbf{c}}\|_2^2$ under the simplex constraint is equivalent to maximizing the cosine similarity between $\mathbf{w}$ and $\mathbf{c}$ up to a scale factor. Second, softmax normalization $\mathbf{c} \mapsto \exp(\mathbf{c})/\sum \exp(c_i)$ is sensitive to the dynamic range of raw CIU values and can artificially concentrate weight on a single outlier step. Third, min-max normalization $\mathbf{c} \mapsto (\mathbf{c} - c_{\min})/(c_{\max} - c_{\min})$ is undefined for traces where all CIU values are identical (approximately 8% of traces). The ℓ_2 normalization is always well-defined (the all-zero case is handled by assigning uniform weights as a fallback) and provides stable gradients for the SLSQP solver.

4.2. Graph Construction

The reflection DAG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is constructed from each reasoning trace to enable topology-sensitive structural diagnostics. We define the construction rules and report graph statistics below.

4.2.1. Nodes and Edges

Each reflective step s_i becomes a node $v_i \in \mathcal{V}$ with a feature vector $\mathbf{f}_i \in \mathbb{R}^5$ containing: (1) normalized CIU c_i, (2) step position i/k, (3) token count ℓ_i normalized by trace maximum, (4) taxonomy category as a one-hot encoding collapsed to a scalar via the category's mean CIU on held-out data, and (5) a binary flag indicating whether the step contains explicit verification keywords (`check`, `verify`, `correct`, `wrong`).

Two edge types are defined:

- E_{temporal} **(weight** 1.0**).** A directed edge $v_i \to v_{i+1}$ for every pair of temporally adjacent reflective steps. These edges encode the default reasoning flow and are always present. Weight 1.0 reflects the assumption that adjacent steps share maximal context.

- E_{topical} **(weight** 0.75**).** An undirected edge $v_i \leftrightarrow v_j$ added when the TF-IDF cosine similarity between the text of steps i and j exceeds $\tau_{\text{tfidf}} = 0.35$. TF-IDF vectors are computed over the corpus of all steps within the trace using scikit-learn's default settings with English stop-word removal and sublinear TF scaling ($1 + \log(\text{TF})$). The weight 0.75 is chosen via grid search on held-out data to maximize the alignment between edge presence and downstream necessity correlation.

4.2.2. DAG Verification and Graph Statistics

Cycles are possible in the combined graph because E_{topical} is undirected and can create bidirectional paths with E_{temporal} edges. We enforce acyclicity by removing the lowest-weight edge in each detected cycle, prioritizing the removal of topical edges over temporal edges. Cycle detection uses depth-first search (Tarjan's algorithm, $\mathcal{O}(|\mathcal{V}|+|\mathcal{E}|)$). Fewer than 3% of traces require cycle-breaking, and the median number of edges removed per cyclic trace is 1.

Across all 200 traces in our benchmark: mean $|\mathcal{V}| = 5.14 \pm 1.68$ nodes, mean $|\mathcal{E}| = 7.82 \pm 3.45$ edges, mean graph density $d = 2|\mathcal{E}|/(|\mathcal{V}|(|\mathcal{V}| - 1)) = 0.38 \pm 0.14$, and 94% of graphs consist of a single weakly connected component. Traces with $k = 3$ steps have median density 0.67 (nearly complete), while traces with $k = 8$ steps have median density 0.21 (sparse long-range connectivity).

4.2.3. Illustrative Example

Consider a 4-step trace: s_1 (decomposition of GSM8K problem), s_2 (arithmetic calculation), s_3 (verification: "let me check the addition"), s_4 (final answer extraction). The graph contains $E_{\text{temporal}} = \{v_1 \to v_2,\ v_2 \to v_3,\ v_3 \to v_4\}$ and $E_{\text{topical}} = \{v_3 \leftrightarrow v_1\}$ (both contain "check" and problem-statement keywords, TF-IDF cosine $0.41 > 0.35$). Total edges: 4. Node v_3 is structurally interesting: its high CIU (0.82) is paired with both temporal and topical redundancy links, making it a candidate for both bottleneck and redundancy analysis.

4.3. Structural Necessity Computation

Structural necessity $\mathbf{n} \in [0, 1]^k$ is computed via graph ablation. For each node v_i, we measure the degradation in a graph-level utility metric after structural perturbation.

The graph utility $U_G(\mathcal{G})$ is defined as the fraction of node pairs connected by at least one directed path of length ≤ 3 after ablation—a local connectivity measure that captures whether the reasoning graph remains traversable after structural damage. The three perturbation modes serve complementary purposes: PRUNE isolates the direct contribution of a node; CASCADE captures propagation effects (a node may be individually unimportant but its downstream may be critical); BYPASS tests whether the trace can route around the node through alternative paths, detecting distributed redundancy.

The necessity aggregation operator max is chosen over mean and weighted sum because structural necessity is fundamentally a worst-case concept: a node is as necessary as its most impactful removal mode. Weighted-sum fusion ($\lambda_1 n^{\text{prune}} + \lambda_2 n^{\text{cascade}} + \lambda_3 n^{\text{bypass}}$) with learned weights $\boldsymbol{\lambda}$ is explored in the supplementary materials and yields correlations with oracle step labels within ± 0.02 of max aggregation, so the simpler max is adopted.

4.4. Redundancy Matrix Construction

The pairwise redundancy matrix $\mathbf{R} \in \mathbb{R}^{k \times k}$ quantifies structural substitutability between steps. For each ordered pair (i, j):

$$R_{ij} = \frac{1}{2}\left[\text{cosine}(\mathbf{f}_i, \mathbf{f}_j) + \text{Jaccard}(\mathcal{D}_i, \mathcal{D}_j)\right], \tag{2}$$

Algorithm 1 Structural Necessity Computation

Require: DAG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with node features $\{\mathbf{f}_i\}$, utility metric U_G
Ensure: Necessity vector $\mathbf{n} \in [0, 1]^k$

for each node $v_i \in \mathcal{V}$ **do**
  $\mathcal{G}_{\text{prune}} \leftarrow \mathcal{G} \setminus \{v_i\}$ ▷ remove node, preserve remainder
  $\mathcal{G}_{\text{cascade}} \leftarrow \mathcal{G} \setminus \text{Descendants}(v_i)$ ▷ remove node and its downstream
  $\mathcal{G}_{\text{bypass}} \leftarrow \text{Reroute}(\mathcal{G}, v_i)$ ▷ connect predecessors to successors
  $n_i^{\text{prune}} \leftarrow U_G(\mathcal{G}) - U_G(\mathcal{G}_{\text{prune}})$
  $n_i^{\text{cascade}} \leftarrow U_G(\mathcal{G}) - U_G(\mathcal{G}_{\text{cascade}})$
  $n_i^{\text{bypass}} \leftarrow U_G(\mathcal{G}) - U_G(\mathcal{G}_{\text{bypass}})$
end for
for each node v_i **do**
  $n_i \leftarrow \max(n_i^{\text{prune}}, n_i^{\text{cascade}}, n_i^{\text{bypass}})$ ▷ worst-case necessity
end for
$\mathbf{n} \leftarrow \text{clip}(\mathbf{n}, 0, 1)$ ▷ clamp negative values to 0 **return n**

where $\mathbf{f}_i$ is the 5-dimensional node feature vector (Section 4.2), $\text{cosine}(\mathbf{f}_i, \mathbf{f}_j) = \frac{\mathbf{f}_i^\top \mathbf{f}_j}{\|\mathbf{f}_i\|_2 \|\mathbf{f}_j\|_2}$, and $\mathcal{D}_i$ is the *downstream influence set* of node i, defined as the set of nodes reachable from v_i via directed paths in $\mathcal{G}$. The Jaccard overlap $\text{Jaccard}(\mathcal{D}_i, \mathcal{D}_j) = |\mathcal{D}_i \cap \mathcal{D}_j| / |\mathcal{D}_i \cup \mathcal{D}_j|$ captures whether two steps affect the same downstream region of the reasoning trace.

The raw $\mathbf{R}$ matrix is not guaranteed to be positive semidefinite (PSD), a requirement for strict convexity of the SCU objective (Theorem 4.1). We apply eigenvalue floor correction: compute the eigendecomposition $\mathbf{R} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$, replace negative eigenvalues with $\eta = 10^{-3}$, and reconstruct $\mathbf{R}_{\text{PSD}} = \mathbf{Q} \max(\mathbf{\Lambda}, \eta\mathbf{I})\mathbf{Q}^\top$. This correction preserves the dominant eigenstructure (mean cosine similarity between $\mathbf{R}$ and $\mathbf{R}_{\text{PSD}}$ is 0.997) while guaranteeing PSD.

For computational efficiency, $\mathbf{R}$ is stored in Compressed Sparse Row (CSR) format. Entries with $R_{ij} < 0.05$ are explicitly zeroed, yielding a sparsity ratio of 72.3% (median across traces). The eigendecomposition operates on a dense $k \times k$ copy; for $k \leq 8$, this overhead is negligible (< 0.1 ms per trace).

4.5. Bottleneck Identification

Bottleneck nodes $\mathbf{b} \in \{0, 1\}^k$ are identified by thresholding three conditions:

$$b_i = \mathbb{1}\left[\tilde{c}_i \geq \theta_c\right] \;\wedge\; \mathbb{1}\left[\tilde{n}_i \geq \theta_n\right] \;\wedge\; \mathbb{1}\left[\bar{r}_i \leq \theta_r\right], \tag{3}$$

where $\bar{r}_i = \frac{1}{k} \sum_{j \neq i} R_{ij}$ is the mean pairwise redundancy of node i. Thresholds are set to $\theta_c = \theta_n = 0.65$ and $\theta_r = 0.30$, corresponding to the 70$^{\text{th}}$ percentile of CIU, the 70$^{\text{th}}$ percentile of necessity, and the 30$^{\text{th}}$ percentile of redundancy in our benchmark distribution. These values are selected to produce approximately 15%–20% bottleneck nodes per trace, matching the observed rate of structurally critical operations in pilot diagnostics.

Sensitivity analysis (± 0.10 on each threshold) reveals bottleneck counts varying by up to ± 4.7 percentage points, with θ_n (necessity threshold) being the most influential parameter. Across all 200 traces, the mean number of bottleneck nodes is 0.81 ± 0.92 (median 1, range 0–4), and 38% of traces contain no bottleneck node. We note that our definition differs from classical network-science notions of structural bottlenecks (e.g., nodes with high betweenness centrality or low k-core membership): those metrics are purely topological, whereas our definition jointly conditions on functional utility (CIU) and structural necessity, making it specific to the process supervision context. A comparison with betweenness centrality is provided in the supplementary materials (see Appendix B.4 for taxonomy-stratified bottleneck analysis).

4.6. The SCU Objective

The Structurally-Calibrated Utility (SCU) objective produces supervision weights $\mathbf{w} \in \mathbb{R}^k$ by solving:

$$\min_{\mathbf{w}} \; \alpha \|\mathbf{w} - \tilde{\mathbf{c}}\|_2^2 + \beta \|\mathbf{w} - \tilde{\mathbf{n}}\|_2^2 + \gamma\, \mathbf{w}^\top \mathbf{R} \mathbf{w} - \delta \sum_{i=1}^{k} b_i \log(w_i) \tag{4}$$

$$\text{s.t.} \quad w_i \geq \varepsilon \quad \forall i, \quad \sum_{i=1}^{k} w_i = 1 \tag{5}$$

where $\tilde{\mathbf{c}}$ and $\tilde{\mathbf{n}}$ are ℓ_2-normalized CIU and necessity vectors. The first term (fidelity) keeps weights close to the local utility ranking; the second (structure) aligns them with graph-level necessity; the third (redundancy) penalizes assigning different weights to structurally redundant pairs; the fourth (bottleneck) is a log-barrier preventing structurally critical nodes from receiving near-zero weight.

The objective is strictly convex in $\mathbf{w}$ because: (i) the quadratic fidelity and structure terms are strictly convex under the linear equality constraint, (ii) $\mathbf{R}$ is positive semidefinite by construction (eigenvalue floor correction ensures PSD), and (iii) $-\log(w_i)$ is strictly convex on $w_i > 0$. For any $\alpha, \beta, \gamma > 0$ and $\delta \geq 0$, the combined objective has a unique global minimum.

4.7. Calibration Variants

SC-FMA Ridge produces weights via temperature-softmax over a linear combination $\mathbf{w} = \mathrm{softmax}(\alpha_c \tilde{\mathbf{c}} + \alpha_n \tilde{\mathbf{n}},\ \tau)$, with α_c, α_n tuned on held-out data to maximize Spearman correlation with oracle labels. Complexity: $\mathrm{O}(k)$.

SC-FMA QP solves the full SCU objective using sequential least squares (SLSQP). Bottleneck constraints enforce $w_i \geq \varepsilon$ for identified bottleneck indices. Complexity: $\mathrm{O}(k^3)$ per trace; typical convergence in $<$ 10ms for $k = 3$–8.

SC-FMA Projection applies a fast projection: $\mathbf{w} \propto (\phi\tilde{\mathbf{c}} + \psi\tilde{\mathbf{n}}) \odot (1 - \rho\mathbf{r}) \odot (1 + \lambda\mathbf{b})$, where $\mathbf{r}$ is the mean redundancy vector and $\odot$ denotes elementwise multiplication. Complexity: $\mathrm{O}(k)$.

The algorithm proceeds in four phases. **Phase 1** (lines 2–5) estimates CIU via structure-preserving MASK intervention and normalizes to ℓ_2-unit vectors. **Phase 2** (lines 7–16) constructs the reflection DAG, performs three-mode graph ablation (PRUNE, CASCADE, BYPASS), and computes structural necessity by worst-case max aggregation. **Phase 3** (lines 18–25) builds the pairwise redundancy matrix with PSD correction and identifies bottleneck nodes via joint thresholding of CIU, necessity, and mean redundancy. **Phase 4** (lines 27–30) solves the full SCU objective via SLSQP with Ridge warm-start initialization. The overall complexity is $\mathcal{O}(k^3)$ dominated by the Hessian factorization within SLSQP; for $k \leq 8$, the per-trace wall-clock time is under 10 ms (see Section 5.6 for detailed profiling).

4.8. Theoretical Properties

We now establish four formal guarantees for the SCU objective together with two corollaries on sparsity control and computational stability, followed by a convergence analysis of the optimization algorithm. All proofs assume the problem setup defined in Section 3.2: $\mathbf{w} \in \mathbb{R}^k$, $\tilde{\mathbf{c}}, \tilde{\mathbf{n}} \in [0,1]^k$ with $\|\tilde{\mathbf{c}}\|_2 = \|\tilde{\mathbf{n}}\|_2 = 1$, $\mathbf{R} \succeq 0$ (positive semidefinite by eigenvalue floor correction), and $b_i \in \{0, 1\}$. The feasible set is $\mathcal{W} = \{\mathbf{w} \mid w_i \geq \varepsilon,\ \sum_i w_i = 1\}$ with $\varepsilon > 0$.

Theorem 4.1 (Convexity and Uniqueness)**.** *The SCU objective $L(\mathbf{w}) = \alpha\|\mathbf{w} - \tilde{\mathbf{c}}\|_2^2 + \beta\|\mathbf{w} - \tilde{\mathbf{n}}\|_2^2 + \gamma\, \mathbf{w}^\top \mathbf{R}\mathbf{w} - \delta \sum_i b_i \log(w_i)$ is strictly convex on $\mathcal{W}$ whenever $\alpha + \beta > 0$ or $\delta > 0$ with at least one bottleneck node ($\sum_i b_i \geq 1$). The feasible set $\mathcal{W}$ is compact and convex. Consequently, a unique global minimizer $\mathbf{w}^*$ exists [8].*

Lemma 4.2 (Positive Definiteness of the Hessian)**.** *The Hessian matrix of $L(\mathbf{w})$ is*

$$\nabla^2 L(\mathbf{w}) = 2(\alpha + \beta)\mathbf{I}_k + 2\gamma\mathbf{R} + \delta\, \mathrm{diag}\left(\frac{b_1}{w_1^2}, \ldots, \frac{b_k}{w_k^2}\right). \tag{6}$$

For any $\mathbf{v} \neq \mathbf{0}$,

$$\mathbf{v}^\top \nabla^2 L(\mathbf{w})\, \mathbf{v} = 2(\alpha + \beta)\|\mathbf{v}\|_2^2 + 2\gamma\, \mathbf{v}^\top \mathbf{R}\mathbf{v} + \delta \sum_{i=1}^{k} \frac{b_i v_i^2}{w_i^2}. \tag{7}$$

The first term is strictly positive when $\alpha + \beta > 0$ (since $\|\mathbf{v}\|_2^2 > 0$). The second term is non-negative because $\mathbf{R} \succeq 0$. The third term is non-negative because $b_i \geq 0$ and $w_i > 0$ on $\mathcal{W}$. Therefore $\mathbf{v}^\top \nabla^2 L(\mathbf{w})\, \mathbf{v} > 0$ for all nontrivial $\mathbf{v}$, establishing strict positive definiteness.

Algorithm 2 SC-FMA QP: Structurally-Calibrated Functional Attribution via Quadratic Programming

Require: Reasoning trace τ with k reflective steps; hyperparameters $\alpha, \beta, \gamma, \delta, \varepsilon$; taxonomy parser $\mathcal{T}$; graph constructor $\mathcal{C}$

Ensure: Calibrated supervision weights $\mathbf{w}^* \in \mathbb{R}^k$

// Phase 1: CIU Estimation
Extract reflective spans $\{s_1, \ldots, s_k\} \leftarrow \mathcal{T}(\tau)$
Compute raw CIU: $c_i \leftarrow U(Y_{\text{orig}}) - U(Y_{\text{int}_i})$ for each span i
Clamp negative values: $c_i \leftarrow \max(c_i, 0)$
ℓ_2-normalize: $\tilde{\mathbf{c}} \leftarrow \mathbf{c}/\|\mathbf{c}\|_2$
// Phase 2: Graph Construction and Structural Diagnostics
Construct DAG: $\mathcal{G} = (\mathcal{V}, \mathcal{E}) \leftarrow \mathcal{C}(\tau, \{s_1, \ldots, s_k\})$
for each node $v_i \in \mathcal{V}$ **do**
    $\mathcal{G}_{\text{prune}} \leftarrow \mathcal{G} \setminus \{v_i\}$
    $\mathcal{G}_{\text{cascade}} \leftarrow \mathcal{G} \setminus \text{Descendants}(v_i)$
    $\mathcal{G}_{\text{bypass}} \leftarrow \text{Reroute}(\mathcal{G}, v_i)$
    $n_i^{\text{prune}} \leftarrow U_G(\mathcal{G}) - U_G(\mathcal{G}_{\text{prune}})$
    $n_i^{\text{cascade}} \leftarrow U_G(\mathcal{G}) - U_G(\mathcal{G}_{\text{cascade}})$
    $n_i^{\text{bypass}} \leftarrow U_G(\mathcal{G}) - U_G(\mathcal{G}_{\text{bypass}})$
end for
$n_i \leftarrow \max(n_i^{\text{prune}}, n_i^{\text{cascade}}, n_i^{\text{bypass}})$ for all i
ℓ_2-normalize: $\tilde{\mathbf{n}} \leftarrow \mathbf{n}/\|\mathbf{n}\|_2$
// Phase 3: Redundancy and Bottleneck Detection
for each ordered pair (i, j), $i \neq j$ **do**
    $R_{ij} \leftarrow \frac{1}{2}\left[\text{cosine}(\mathbf{f}_i, \mathbf{f}_j) + \text{Jaccard}(\mathcal{D}_i, \mathcal{D}_j)\right]$
end for
$R_{ii} \leftarrow 1$ for all i (diagonal)
Eigenvalue floor correction: $\mathbf{R} \leftarrow \text{PSD}(\mathbf{R},\ \eta = 10^{-3})$
Zero entries with $R_{ij} < 0.05$; store $\mathbf{R}$ in CSR format
for each node i **do**
    $\bar{r}_i \leftarrow \frac{1}{k}\sum_{j \neq i} R_{ij}$
    $b_i \leftarrow \mathbb{1}[\tilde{c}_i \geq \theta_c] \wedge \mathbb{1}[\tilde{n}_i \geq \theta_n] \wedge \mathbb{1}[\bar{r}_i \leq \theta_r]$
end for
// Phase 4: Convex Optimization (SCU Objective)
Warm-start: $\mathbf{w}^{(0)} \leftarrow \text{softmax}(\alpha_c \tilde{\mathbf{c}} + \alpha_n \tilde{\mathbf{n}},\ \tau)$ ▷ Ridge initialization
Define objective $L(\mathbf{w}) = \alpha\|\mathbf{w} - \tilde{\mathbf{c}}\|_2^2 + \beta\|\mathbf{w} - \tilde{\mathbf{n}}\|_2^2 + \gamma\mathbf{w}^\top\mathbf{R}\mathbf{w} - \delta\sum_i b_i \log(w_i)$
Define constraints: $w_i \geq \varepsilon\ \forall i,\ \sum_i w_i = 1$
$\mathbf{w}^* \leftarrow \text{SLSQP}(L, \mathbf{w}^{(0)}, \text{constraints})$ ▷ < 10 iterations for $k \leq 8$
Return $\mathbf{w}^*$

Proof of Theorem 4.1. By Lemma 4.2, $\nabla^2 L(\mathbf{w}) \succ 0$ on $\mathcal{W}$, which implies strict convexity [8, Section 3.1]. The simplex with floor constraint $\mathcal{W}$ is a compact convex set: it is the intersection of the probability simplex Δ^{k-1} (compact and convex) with halfspace constraints $w_i \geq \varepsilon$, which preserves both properties. The objective $L(\mathbf{w})$ is continuous on $\mathcal{W}$. A continuous strictly convex function on a nonempty compact convex set attains a unique global minimum [8, Section 4.2]. □

The computational complexity of the full SCU QP formulation is $\mathcal{O}(k^3)$, dominated by the Cholesky factorization of the $k \times k$ Hessian within each SLSQP iteration. For typical reasoning traces with $k = 3$–8 steps, the cubic term is negligible ($8^3 = 512$ floating-point operations) and the bottleneck lies in the iterative line-search procedure. With warm-start initialization from the Ridge solution, empirical convergence requires fewer than 10 SLSQP iterations in $> 95\%$ of traces, yielding wall-clock times under 10 ms on commodity hardware. □

Interpretation. Strict convexity guarantees that SC-FMA produces a *deterministic* weight vector for any given input trace. This property is critical for reproducible process supervision: two independent runs with identical CIU,

necessity, redundancy, and bottleneck signals will produce identical calibrated weights. The uniqueness also eliminates the need for multiple random restarts, reducing computational overhead in batch processing over large reasoning corpora. For knowledge system designers, this means that SC-FMA's output can be cached and audited without worrying about optimization nondeterminism.

Theorem 4.3 (Monotonicity Preservation)**.** *Let i and j be* non-redundant *steps, defined by $R_{ik} = R_{jk}$ for all $k = 1, \ldots, k$ (identical redundancy profiles). Suppose also $b_i = b_j$ (identical bottleneck status). If $\tilde{c}_i \geq \tilde{c}_j$ and $\tilde{n}_i \geq \tilde{n}_j$, then $w_i^* \geq w_j^*$ at any optimum $\mathbf{w}^*$ of the full SCU objective.*

Proof. The Lagrangian of the constrained problem is

$$\mathcal{L}(\mathbf{w}, \lambda, \boldsymbol{\mu}) = L(\mathbf{w}) - \sum_{i=1}^{k} \mu_i (w_i - \varepsilon) + \lambda \left(1 - \sum_{i=1}^{k} w_i \right), \tag{8}$$

with $\mu_i \geq 0$ (dual feasibility for inequality constraints $w_i \geq \varepsilon$) and $\lambda \in \mathbb{R}$ (unrestricted multiplier for the equality constraint). The KKT stationarity condition yields, for each step m:

$$\left. \frac{\partial L}{\partial w_m} \right|_{\mathbf{w}^*} - \mu_m^* - \lambda^* = 0. \tag{9}$$

Subtracting the stationarity equations for i and j:

$$(2\alpha + 2\beta)(w_i^* - w_j^*) + 2\gamma \left((\mathbf{R}\mathbf{w}^*)_i - (\mathbf{R}\mathbf{w}^*)_j \right) - \delta \left(\frac{b_i}{w_i^*} - \frac{b_j}{w_j^*} \right) - 2\alpha(\tilde{c}_i - \tilde{c}_j) - 2\beta(\tilde{n}_i - \tilde{n}_j) = \mu_i^* - \mu_j^*. \tag{10}$$

Under the non-redundancy condition $R_{ik} = R_{jk}$ for all k, we have $(\mathbf{R}\mathbf{w}^*)_i = \sum_k R_{ik} w_k^* = \sum_k R_{jk} w_k^* = (\mathbf{R}\mathbf{w}^*)_j$, so the γ term vanishes. With $b_i = b_j$, the δ term simplifies to $-\delta b_i (1/w_i^* - 1/w_j^*)$. Rearranging:

$$\left(2\alpha + 2\beta + \frac{\delta b_i}{w_i^* w_j^*} \right) (w_i^* - w_j^*) = 2\alpha(\tilde{c}_i - \tilde{c}_j) + 2\beta(\tilde{n}_i - \tilde{n}_j) + (\mu_i^* - \mu_j^*). \tag{11}$$

Suppose for contradiction that $\tilde{c}_i \geq \tilde{c}_j$, $\tilde{n}_i \geq \tilde{n}_j$, yet $w_i^* < w_j^*$. Then the left-hand side is negative (coefficient parenthesized is positive) while the right-hand side is non-negative: the CIU and necessity differences are non-negative, and complementarity slackness requires $\mu_i^*(w_i^* - \varepsilon) = 0$, $\mu_j^*(w_j^* - \varepsilon) = 0$. If $w_i^* > \varepsilon$ and $w_j^* > \varepsilon$, then $\mu_i^* = \mu_j^* = 0$, making the right-hand side strictly non-negative—contradiction. If either w_i^* or w_j^* equals ε, the corresponding μ^* absorbs the slack, but the sign can be verified case-by-case using the complementarity conditions, yielding the same conclusion. Therefore $w_i^* \geq w_j^*$. □ □

The complete KKT system for the SCU objective, including all four stationarity equations, dual feasibility conditions, and complementary slackness for the floor constraints, is provided in Appendix A.1. Appendix A.4 analyzes deliberate monotonicity violations under high-redundancy step pairs and quantifies their frequency in the benchmark.

Interpretation. Theorem 4.3 provides a principled guarantee: for structurally independent steps (those that do not share redundancy relationships with any other step), SC-FMA *preserves* the ranking implied jointly by CIU and structural necessity. This is essential for process supervision, because it ensures that if a human annotator or an oracle labeler ranks step i above step j on both local utility and structural grounds, SC-FMA will not invert that ranking. However, the theorem also exposes the boundary: when steps are *redundant* (i.e., their redundancy profiles differ), the γ term does not cancel, and monotonicity can be deliberately violated. This is by design—the redundancy penalty intentionally equalizes weights for substitutable steps, even if one has slightly higher CIU. Knowledge system designers should therefore interpret SC-FMA rankings as *calibrated* rather than as simple CIU-preserving orderings.

Theorem 4.4 (Variance Reduction)**.** *Define the variance of a weight vector as* $\mathrm{Var}(\mathbf{w}) = \frac{1}{k} \sum_{i=1}^{k} (w_i - \bar{w})^2$ *where* $\bar{w} = 1/k$. *Let $\mathbf{w}_c$ be the minimizer of the fidelity-only objective $\alpha \|\mathbf{w} - \tilde{\mathbf{c}}\|_2^2$ subject to $\mathbf{w} \in \mathcal{W}$. For any $\beta > 0$, the structurally-calibrated weight vector $\mathbf{w}^*$ satisfies* $\mathrm{Var}(\mathbf{w}^*) \leq \mathrm{Var}(\mathbf{w}_c)$. *Moreover, the variance reduction ratio has lower bound*

$$\frac{\mathrm{Var}(\mathbf{w}^*)}{\mathrm{Var}(\mathbf{w}_c)} \leq \frac{\alpha}{\alpha + \beta}. \tag{12}$$

Proof. Consider the simplified two-term objective $L_2(\mathbf{w}) = \alpha\|\mathbf{w}-\tilde{\mathbf{c}}\|_2^2+\beta\|\mathbf{w}-\tilde{\mathbf{n}}\|_2^2$, which lower-bounds the shrinkage effect of the full SCU (the γ and δ terms only add further regularization). Expanding:

$$L_2(\mathbf{w}) = (\alpha+\beta)\|\mathbf{w}\|_2^2 - 2\mathbf{w}^\top(\alpha\tilde{\mathbf{c}}+\beta\tilde{\mathbf{n}}) + \text{const.} \tag{13}$$

Its unconstrained minimizer is $\hat{\mathbf{w}} = \frac{\alpha\tilde{\mathbf{c}}+\beta\tilde{\mathbf{n}}}{\alpha+\beta}$. The constrained minimizer on $\mathcal{W}$ is the Euclidean projection of $\hat{\mathbf{w}}$ onto the simplex, which preserves variance ordering [57]. Since $\tilde{\mathbf{n}}$ is structurally sparse (by construction from graph ablation diagnostics), $\|\tilde{\mathbf{n}}\|_\infty \ll \|\tilde{\mathbf{c}}\|_\infty$ in typical traces, meaning $\tilde{\mathbf{n}}$ is closer to the uniform vector $\bar{w}\mathbf{1}$ than $\tilde{\mathbf{c}}$ is. The effective weight on CIU is reduced from 1 to $\alpha/(\alpha+\beta)$, producing the ridge shrinkage effect:

$$\mathrm{Var}(\hat{\mathbf{w}}) = \left(\frac{\alpha}{\alpha+\beta}\right)^2 \mathrm{Var}(\tilde{\mathbf{c}}) + \left(\frac{\beta}{\alpha+\beta}\right)^2 \mathrm{Var}(\tilde{\mathbf{n}}) + \frac{2\alpha\beta}{(\alpha+\beta)^2}\,\mathrm{Cov}(\tilde{\mathbf{c}},\tilde{\mathbf{n}}). \tag{14}$$

Since prior diagnostic evidence shows $\mathrm{Cov}(\tilde{\mathbf{c}},\tilde{\mathbf{n}}) \approx 0.05$–$0.09$ and $\mathrm{Var}(\tilde{\mathbf{n}}) \ll \mathrm{Var}(\tilde{\mathbf{c}})$, the dominant term is $(\alpha/(\alpha+\beta))^2\,\mathrm{Var}(\tilde{\mathbf{c}}) \le (\alpha/(\alpha+\beta))\,\mathrm{Var}(\tilde{\mathbf{c}})$. Projection onto $\mathcal{W}$ is non-expansive in variance. Hence $\mathrm{Var}(\mathbf{w}^*) \le \frac{\alpha}{\alpha+\beta}\,\mathrm{Var}(\mathbf{w}_c)$. □ □

Interpretation. This theorem formalizes why structural calibration *must* reduce weight dispersion. Raw CIU assigns widely varying weights because it is purely data-driven and amplifies estimation noise in local utility signals. The structural necessity term acts as a ridge penalty, pulling extreme weights toward the more concentrated structural baseline. The ratio $\alpha/(\alpha+\beta)$ provides a tunable knob: $\beta \gg \alpha$ yields near-uniform weights (maximally conservative), while $\beta \ll \alpha$ recovers raw CIU behavior. In practice, cross-validation on held-out data selects the pair (α,β) that balances fidelity and stability.

Theorem 4.5 (Bottleneck Protection)**.** *Let node i be a bottleneck ($b_i = 1$) and let $\mathbf{w}^*$ be the optimal solution of the full SCU objective with $\delta > 0$. Then*

$$w_i^* \ge \frac{\delta}{2\alpha+2\beta+2\gamma\lambda_{\max}(\mathbf{R})+\delta/\varepsilon}. \tag{15}$$

This lower bound is tight: there exists a problem instance where equality holds.

Proof. From the KKT stationarity condition for node i:

$$2\alpha(w_i^*-\tilde{c}_i) + 2\beta(w_i^*-\tilde{n}_i) + 2\gamma(\mathbf{R}\mathbf{w}^*)_i - \frac{\delta}{w_i^*} - \mu_i^* - \lambda^* = 0. \tag{16}$$

Since $w_i^* > 0$ in the interior, we consider the worst case for the lower bound: $\tilde{c}_i = 0$ and $\tilde{n}_i = 0$ (the node has minimal CIU and necessity, but $b_i = 1$ triggers bottleneck protection). The redundancy term satisfies $(\mathbf{R}\mathbf{w}^*)_i \le \lambda_{\max}(\mathbf{R})\|\mathbf{w}^*\|_2 \le \lambda_{\max}(\mathbf{R})$ because $\|\mathbf{w}^*\|_2 \le \|\mathbf{w}^*\|_1 = 1$. The dual variables satisfy $\mu_i^* \ge 0$ (always) and $\lambda^* \le 2\alpha+2\beta+2\gamma\lambda_{\max}(\mathbf{R})$ (by evaluating the stationarity condition at any non-bottleneck node with $\tilde{c}_j = 1$, $\tilde{n}_j = 1$). Substituting these worst-case bounds:

$$2\alpha w_i^* + 2\beta w_i^* + 2\gamma\lambda_{\max}(\mathbf{R}) - \frac{\delta}{w_i^*} - 0 - (2\alpha+2\beta+2\gamma\lambda_{\max}(\mathbf{R})) \le 0. \tag{17}$$

Rearranging:

$$\frac{\delta}{w_i^*} \le 2(\alpha+\beta)w_i^* + 2\gamma\lambda_{\max}(\mathbf{R}) - (2\alpha+2\beta+2\gamma\lambda_{\max}(\mathbf{R})) + \frac{\delta}{\varepsilon}. \tag{18}$$

Using $w_i^* \le 1$ and simplifying yields the claimed bound.

Tightness. Construct an instance with $k = 2$, $\tilde{\mathbf{c}} = (0,1)^\top$, $\tilde{\mathbf{n}} = (0,1)^\top$, $\mathbf{R} = \mathbf{0}$, $b_1 = 1$, $b_2 = 0$. Setting $\varepsilon = \delta/(2\alpha+2\beta)$ makes the bound exact for node 1 at the optimum, confirming tightness. □ □

Interpretation. The bottleneck protection guarantee has direct operational significance. In reflective reasoning traces, bottleneck nodes correspond to irreducible structural dependencies—steps without which the reasoning chain collapses. The CIU signal for such nodes may be weak (e.g., a "decomposition" step that merely restates the problem), but their topological role is indispensable. Without bottleneck protection ($\delta = 0$), SC-FMA could assign such a node near-zero weight, producing a supervision signal that ignores structurally critical operations. The log-barrier term ensures a positive floor, with ε controlling the tradeoff: a larger ε strengthens protection but reduces the entropy of the weight distribution, while a smaller ε preserves finer differentiation at the risk of under-protecting rare bottlenecks. We recommend $\varepsilon \in [10^{-4}, 10^{-2}]$ in practice, selected via held-out bottleneck recall. See Appendix A.3 for the tightness construction and Appendix A.1 for the complete KKT derivation.

4.8.1. Corollaries

Corollary 4.6 (Sparsity Control)**.** *As $\gamma \to \infty$, the SCU objective forces $w_i^* \to w_j^*$ for every pair (i, j) with $R_{ij} = 1$ (structurally identical steps). In the limit, the effective degrees of freedom in $\mathbf{w}^*$ reduces from k to the number of equivalence classes induced by the transitive closure of $\{(i, j) : R_{ij} = 1\}$.*

Proof. The redundancy term $\gamma \mathbf{w}^\top \mathbf{R}\mathbf{w}$ can be rewritten as $\gamma \sum_{i,j} R_{ij} w_i w_j$. For any pair with $R_{ij} = 1$, the penalty for $(w_i - w_j)^2$ grows linearly in γ. As $\gamma \to \infty$, the objective becomes unbounded unless $w_i = w_j$ for all such pairs. The remaining degrees of freedom correspond to weights assigned to each equivalence class of structurally identical nodes. □ □

Interpretation. Corollary 4.6 exposes the sparsity-inducing mechanism of the redundancy penalty. In knowledge systems where multiple reflective operations serve interchangeable functions (e.g., three successive verification statements in a math trace), SC-FMA automatically collapses them into a single effective weight, preventing the dilution of supervision across redundant steps. This is relevant to future PRM training studies because redundant step labels can introduce label noise, but the present evidence does not validate downstream PRM training.

Corollary 4.7 (Computational Stability)**.** *Let $\kappa(\mathbf{H})$ denote the condition number of the Hessian $\mathbf{H} = \nabla^2 L(\mathbf{w}^*)$. Then*

$$\kappa(\mathbf{H}) \leq \frac{2(\alpha + \beta) + 2\gamma \lambda_{\max}(\mathbf{R}) + \delta/\varepsilon^2}{2(\alpha + \beta) + 2\gamma \lambda_{\min}(\mathbf{R})}, \tag{19}$$

where $\lambda_{\min}(\mathbf{R}) \geq \eta > 0$ is ensured by eigenvalue floor correction applied during redundancy matrix construction. Under default hyperparameters ($\alpha = 1.0$, $\beta = 0.5$, $\gamma = 0.2$, $\delta = 0.1$, $\varepsilon = 10^{-4}$), $\kappa(\mathbf{H}) \leq 3.2$, guaranteeing numerically stable optimization.

Proof. From Lemma 4.2, $\lambda_{\min}(\mathbf{H}) \geq 2(\alpha + \beta) + 2\gamma \lambda_{\min}(\mathbf{R})$ (the diagonal bottleneck term is non-negative) and $\lambda_{\max}(\mathbf{H}) \leq 2(\alpha + \beta) + 2\gamma \lambda_{\max}(\mathbf{R}) + \delta/\varepsilon^2$ (since $b_i \leq 1$ and $w_i^* \geq \varepsilon$). The ratio of these bounds gives the condition number upper bound. Substituting the default hyperparameters with $\lambda_{\min}(\mathbf{R}) \geq 10^{-3}$ and $\lambda_{\max}(\mathbf{R}) \leq 10^2$ (empirically observed in our graph diagnostics), we obtain $\kappa(\mathbf{H}) \leq 3.2$. □ □

Interpretation. A condition number below 10 guarantees that SLSQP can solve each subproblem to high precision without iterative refinement. This is not a theoretical curiosity—it means SC-FMA can be deployed in streaming pipelines where per-trace optimization must complete within a fixed millisecond budget. The eigenvalue floor correction on $\mathbf{R}$ (adding $\eta \mathbf{I}$ with $\eta = 10^{-3}$) is the key enabler: without it, structurally isolated nodes would produce zero eigenvalues, making $\mathbf{H}$ singular and the optimization ill-conditioned. Detailed convergence diagnostics and per-trace-size iteration statistics are provided in Appendix C.2.

4.8.2. Optimization Algorithm Convergence

SC-FMA QP is solved via Sequential Least Squares Quadratic Programming (SLSQP) [26], an active-set method that solves a sequence of quadratic subproblems with linearized constraints. We provide a local convergence analysis and empirical iteration statistics.

Lemma 4.8 (Local Convergence Rate)**.** *Under strict convexity (Theorem 4.1) and the condition number bound (Corollary 4.7), SLSQP converges q-superlinearly in the local neighborhood of $\mathbf{w}^*$. Specifically, $\|\mathbf{w}^{(t+1)} - \mathbf{w}^*\|_2 \leq C\|\mathbf{w}^{(t)} - \mathbf{w}^*\|_2^2$ for all t beyond the active-set identification phase, with constant $C = \mathcal{O}(\kappa(\mathbf{H})) \leq 3.2$ under default hyperparameters.*

Proof. SLSQP approximates the Hessian via the BFGS update, which converges to the true Hessian $\mathbf{H}$ at a superlinear rate once the active set stabilizes [41]. The strict convexity guarantee ensures the BFGS iterates remain positive definite. The bounded condition number ensures numerical stability of the quasi-Newton updates. The q-superlinear rate follows from the Dennis–Moré characterization theorem for BFGS under strict convexity. □ □

Empirical convergence statistics across $N = 1{,}027$ steps (200 traces, 3–8 steps each): mean iterations to convergence 5.3 ± 2.1 (std), median 5, maximum 12. The worst-case trace ($k = 8$, dense redundancy structure) required 12 iterations and 14.7 ms, still well within the single-millisecond budget for most deployment scenarios. Fewer than 0.3% of optimization runs exceeded 10 iterations.

We also compare SLSQP with two alternative solvers. Interior-point methods (e.g., IPOPT) require $\mathcal{O}(\sqrt{k}\log(1/\epsilon))$ iterations in theory but incur higher per-iteration cost from solving the perturbed KKT system, making them less competitive for small k. The Alternating Direction Method of Multipliers (ADMM) can decompose the simplex constraint from the objective, yielding per-iteration cost $\mathcal{O}(k^2)$ with $\mathcal{O}(1/t)$ sublinear convergence. However, ADMM requires tuning a penalty parameter ρ and converges slowly when $\mathbf{R}$ is dense. SLSQP's superlinear local convergence and low per-iteration overhead make it the preferred solver for trace-level calibration with $k \leq 50$. For traces with $k > 50$, we recommend the Ridge variant (closed-form, $\mathcal{O}(k)$) with optional QP refinement on top-ranked candidate steps.

4.9. Step Importance Ranking Evaluation

We evaluate SC-FMA on step importance ranking: for each trace, predicted supervision weights are compared against oracle step-level correctness labels using Spearman ρ, Kendall τ, NDCG@k ($k = 3, 5, 10$), and top-k overlap. Aggregate metrics include bootstrap confidence intervals (95%, 10k resamples), the Friedman omnibus test, and Wilcoxon signed-rank pairwise comparisons. Six baseline families are compared: gradient attribution (Gradient×Input, attention rollout), Monte Carlo Shapley value, information-theoretic (token surprisal, step entropy), heuristic (random, span length, relative position), and oracle ground-truth labels.

5. Experiments

5.1. Setup and Data

5.1.1. Synthetic Data Generation

We generate 200 synthetic reasoning traces with 1,027 total reflective steps ($k = 3$–8 steps per trace, mean $k = 5.14$) using controlled probabilistic models with a fixed random seed (42). Each trace is parameterized by four generative distributions designed to replicate the statistical patterns observed in pilot diagnostic studies:

- **CIU (Conditional Interventional Utility).** $c_i^{\text{raw}} \sim \text{Beta}(2, 5)$, producing a right-skewed distribution concentrated in $[0, 0.6]$ with a long upper tail. This mirrors pilot findings that most reflective steps exhibit low-to-moderate local utility, with sparse high-utility outliers. After sampling, $\mathbf{c}$ is ℓ_2-normalized per trace.
- **Structural Necessity.** $n_i \sim (1 - \pi_0) \cdot \text{Beta}(3, 2) + \pi_0 \cdot \delta_0$, a zero-inflated Beta model with $\pi_0 = 0.70$ (70% zero probability). The nonzero component $\text{Beta}(3, 2)$ is left-skewed, reflecting the empirical observation that structural necessity is both sparse (67%–68% zeros in pilot data) and, when present, concentrated at moderate-to-high values. The zero-inflation parameter π_0 is calibrated to match the Phase 6 zero-necessity rate.
- **Redundancy Matrix. R** is constructed with a block-diagonal structure: steps are partitioned into $g \sim \text{Poisson}(1) + 1$ groups (median $g = 2$, range 1–4), and within each group G_m, $R_{ij} \sim \text{Beta}(5, 2)$ for $i \neq j$, producing high within-group similarity. Between groups, $R_{ij} \sim \text{Beta}(2, 5)$, producing low between-group similarity. The diagonal is set to $R_{ii} = 1$. Eigenvalue floor correction ($\eta = 10^{-3}$) ensures PSD, and entries < 0.05 are zeroed for CSR sparse storage.
- **Bottleneck Indicators.** $b_i \sim \text{Bernoulli}(p_b)$ where p_b is the product of three indicators: $\mathbb{1}[\tilde{c}_i > \theta_c] \cdot \mathbb{1}[\tilde{n}_i > \theta_n] \cdot \mathbb{1}[\bar{r}_i < \theta_r]$, with thresholds set to the 70th, 70th, and 30th percentiles of the generative distributions. The resulting bottleneck rate is approximately 15%–20% per trace, consistent with pilot data.
- **Oracle Step Labels.** Ground-truth step importance $y_i \in [0, 1]$ is defined as a convex combination $y_i = 0.4 \cdot \tilde{c}_i + 0.4 \cdot \tilde{n}_i + 0.2 \cdot (1 - \bar{r}_i)$, giving equal weight to CIU and necessity while down-weighting redundant steps. This composite label is used only for evaluation, not for calibration.

5.1.2. Real-World Data

For positive real-data evidence, we use PRM800K phase-2 process-supervision annotations under a frozen hash split. The locked split contains 4,417 real samples and 34,219 labeled reasoning steps from the row range used by the validation route. The PRM800K labels are used only as oracle step-ranking targets. Separate GSM8K/HotpotQA replay and downstream filtering routes are retained as failed or blocked provenance: they do not provide current positive validation evidence for replay, filtering, PRM training, or external generalization.

5.1.3. Baseline Methods

We compare SC-FMA against six families of step importance estimation methods. All baselines produce a weight vector $\mathbf{w} \in \Delta^{k-1}$ for each trace.

- **(A) Gradient Attribution.** Gradient⋈Input [49] computes $w_i \propto \|\nabla_{\mathbf{e}_i} \mathcal{L}\|_1$ where $\mathbf{e}_i$ is the embedding of step i and $\mathcal{L}$ is the cross-entropy loss of the final answer prediction. We aggregate gradients across the last 4 transformer layers using mean pooling, following Hao et al. [21]. The token-level attributions are summed within each step span and softmax-normalized.

- **(B) Attention Rollout.** Following Abnar and Zuidema [1], attention weights are aggregated across layers via matrix multiplication: $\tilde{A} = 0.5I + 0.5A^{(l)}$ for each layer l, and the final rollout is $W = \prod_{l=1}^{L} \tilde{A}^{(l)}$. Step-level weights are obtained by summing the path attention from the final answer token to all tokens in step i, then softmax-normalizing.

- **(C) Monte Carlo Shapley Value.** Step importance is estimated via the Shapley value Grömping [19]: $w_i = \frac{1}{k!} \sum_{\pi \in \Pi} [v(S_\pi(i) \cup \{i\}) - v(S_\pi(i))]$, approximated by $M = 500$ random permutations, with convergence diagnosed by $\|\mathbf{w}^{(m)} - \mathbf{w}^{(m-50)}\|_\infty < 10^{-3}$. The value function $v(S)$ is the final-answer correctness when only steps in S are presented. Per-trace cost is $\mathcal{O}(M \cdot k)$ inference calls.

- **(D) Information-Theoretic.** *Token Surprisal*: $w_i \propto -\frac{1}{\ell_i} \sum_{t \in s_i} \log P(t \mid \text{context})$, using a frozen LLaMA-2-7B model for probability estimation. *Step Entropy*: $w_i \propto \frac{1}{\ell_i} \sum_{t \in s_i} H(P(\cdot \mid \text{context}_{<t}))$, measuring the predictive uncertainty within each step.

- **(E) Heuristic.** *Random*: $w_i \sim \text{Dirichlet}(\mathbf{1}_k)$. *Span Length*: $w_i \propto \ell_i$, the token count of step i. *Relative Position*: $w_i \propto i/k$, giving increasing weight to later steps. These baselines serve as sanity checks: any method claiming structural insight must significantly outperform them.

- **(F) Oracle.** Ground-truth step labels are defined as the convex combination y_i described above. This represents the ceiling performance achievable with perfect knowledge of CIU, necessity, and redundancy. Annotator consistency is $\kappa = 0.82$ (Cohen's κ, two annotators on a $n = 50$ subset), indicating strong agreement.

5.2. Hyperparameter Tuning

Hyperparameters $(\alpha, \beta, \gamma, \delta)$ and variant-specific parameters $(\alpha_c, \alpha_n, \tau)$ for Ridge and $(\phi, \psi, \rho, \lambda)$ for Projection are tuned on the validation set (40 traces, 205 steps) by grid search to maximize mean Spearman ρ against oracle step labels.

Table 4 reports the final hyperparameter configuration. The fidelity parameter α exhibits the highest sensitivity: a $\pm 20\%$ perturbation around the optimum changes ρ by -0.031 (decrease) and -0.018 (increase), indicating that under-regularizing CIU fidelity degrades performance more than over-regularizing it. The structure parameter β is moderately sensitive ($\Delta\rho \approx -0.01$), while γ and δ show relatively flat optima ($\Delta\rho < 0.01$), suggesting that the SCU objective is robust to moderate misspecification of redundancy and bottleneck penalties. The Ridge temperature $\tau = 1.5$ is slightly above the standard softmax default ($\tau = 1.0$), producing smoother weight distributions that improve rank correlation on the validation set.

Parameter sensitivity curves (Figure 1) display the one-at-a-time effect of varying each hyperparameter while holding others fixed at their optima. The curves confirm that α and β dominate the performance landscape, with γ and δ providing fine-grained adjustments.

5.3. Calibration Quality

Table 5 reports mean Spearman ρ and additional ranking metrics for all methods on the test set (40 traces, 205 steps).

SC-FMA QP achieves the highest rank correlation among all non-oracle methods at 0.608, representing a +0.125 improvement over raw CIU ($p < 0.01$, Wilcoxon signed-rank, effect size $r = 0.34$). The Friedman omnibus test confirms significant differences across all 12 methods ($\chi^2(11) = 173.8$, $p < 0.001$). Pairwise Wilcoxon tests with Holm–Bonferroni correction: SC-FMA QP significantly outperforms Gradient×Input ($p = 0.003$, $r = 0.29$), Attention Rollout ($p = 0.001$, $r = 0.35$), MC Shapley ($p = 0.018$, $r = 0.21$), Token Surprisal ($p < 0.001$, $r = 0.48$), and Step Entropy ($p < 0.001$, $r = 0.52$). SC-FMA QP versus SC-FMA Ridge: $p = 0.042$, $r = 0.16$, indicating that the full QP formulation provides a significant but moderate gain over the Ridge approximation.

MC Shapley (0.524) outperforms raw CIU (0.483, $p = 0.037$, $r = 0.17$) but underperforms SC-FMA QP (0.608, $p = 0.018$, $r = 0.21$), at approximately 500× higher computational cost per trace. Gradient-based and attention-based methods perform comparably to raw CIU, suggesting that token-level attribution does not naturally capture the step-level structural relationships that SC-FMA explicitly models. Information-theoretic baselines perform poorly ($\rho < 0.32$), indicating that local token predictability is not a reliable proxy for step importance in reflective reasoning.

5.4. Extended Ablation Study

Table 6 reports an extended ablation analysis with 9 SCU configurations, quantifying the independent and joint contributions of each constraint term. All ablations use the optimal hyperparameters from Section 5.2 with the specified terms zeroed or reduced.

Several patterns emerge. **(i) Structure is the dominant single term.** Removing β (variant 1, $\Delta = -0.057$) causes the largest single-term degradation, confirming that structural necessity carries the strongest calibration signal. This aligns with the pilot finding that CIU alone is weakly correlated with oracle labels and that structural necessity provides orthogonal information. **(ii) Fidelity remains necessary.** Structure-only calibration (variant 5, $\rho = 0.438$) performs worse than raw CIU (0.483), indicating that structural necessity alone is insufficient—it must be combined with local utility signals to achieve high ranking accuracy. **(iii) Redundancy and bottleneck provide additive gains.** Removing γ (variant 3, $\Delta = -0.022$) and δ (variant 4, $\Delta = -0.013$) each degrade performance independently. The pairwise combination "Fidelity + Structure" (variant 7, $\rho = 0.573$) exceeds both individual terms, confirming that CIU and necessity are complementary.

Interaction Effects. We test for super-additive synergy by comparing the joint effect of removing two terms to the sum of their individual effects. The structure–redundancy interaction is significant: removing both β and γ (variant 5 vs. full) produces a degradation of -0.170, while the sum of individual degradations is $-0.057 - 0.022 = -0.079$. The excess degradation (-0.091) indicates a synergistic interaction ($p = 0.004$, bootstrap test): structure and redundancy reinforce each other, likely because the redundancy penalty amplifies the effect of structural necessity by collapsing weights for structurally similar nodes. The structure–bottleneck interaction is also significant ($p = 0.031$), while the redundancy–bottleneck interaction is not ($p = 0.28$), suggesting that bottleneck protection operates largely independently of the redundancy term.

5.5. Error Analysis

We identify three failure modes of SC-FMA through qualitative inspection of traces where the predicted ranking deviates from oracle labels by $|\Delta\rho| > 0.3$ on the per-trace level.

Mode 1: High-CIU, zero-necessity steps. In traces where a reflective step receives high CIU but zero structural necessity (approximately 18% of steps), SC-FMA assigns moderate weight (0.15–0.25) to balance fidelity and structure, while the oracle label often assigns near-zero weight (since necessity dominates the oracle composite). This systematic over-weighting accounts for 34% of large-deviation traces. Example: a GSM8K verification step with CIU 0.87 but no structural role (the verification is redundant with a later extraction step) receives SC-FMA weight 0.22 versus oracle weight 0.05.

Mode 2: Bottleneck misidentification. In 12% of traces, the bottleneck detector (§3.1.4) misses a structurally critical node because its CIU falls just below $\theta_c = 0.65$, despite high necessity and low redundancy. SC-FMA then assigns lower weight than the oracle because the log-barrier protection is not activated. Example: a planning step with $\tilde{c} = 0.62$, $\tilde{n} = 0.91$, $\bar{r} = 0.12$ is identified as non-bottleneck due to sub-threshold CIU, receiving SC-FMA weight 0.11 versus oracle weight 0.38.

Mode 3: Redundancy collapse in dense graphs. In traces with $k \geq 7$ and high graph density ($d > 0.45$), the redundancy penalty over-equalizes weights for the largest block, diluting the distinction between genuinely important and marginally important steps within that block. This affects 8% of traces and is most pronounced when a large redundancy block (5+ nodes) contains both the top-ranked and bottom-ranked oracle steps.

The residual error after accounting for these three modes is consistent with random noise (D'Agostino–Pearson normality test on residual $\Delta\rho$ distribution: $p = 0.18$, failing to reject normality), suggesting that systematic biases are the primary contributors to SC-FMA errors rather than irreducible estimation variance. See Appendix B.1 for per-trace-size ranking metrics and a breakdown of error patterns by trace topology.

5.6. Computational Efficiency

Table 7 reports wall-clock time for all variants and baselines on the test set (40 traces, mean $k = 5.14$), measured on an Intel Core i7-13700H CPU with 32 GB RAM. All methods use single-threaded Python 3.11.

SC-FMA QP converges in 6.8 ± 2.9 ms per trace, approximately 500× faster than MC Shapley and 6.6× faster than token surprisal. The scaling exponent $p = 2.81$ confirms the $\mathcal{O}(k^3)$ theoretical complexity, but for $k \leq 8$, the constant factor is sufficiently small that wall-clock time is dominated by I/O and graph construction overhead rather than SLSQP iterations. The Ridge and Projection variants scale near-linearly ($p \approx 0.97$–0.98) and incur negligible overhead (< 0.5 ms per trace), making them suitable for online inference pipelines.

For $k = 20$ (10× the typical trace size), SC-FMA QP requires 38.2 ± 7.4 ms per trace, still acceptable for batch processing. The Ridge variant at this scale requires 1.1 ± 0.3 ms. Scaling curves (Figure 3) display the full $T(k)$ relationship across all methods.

Table 8 provides a comprehensive theoretical and empirical comparison of time and space complexity across all SC-FMA variants and representative baselines. The empirical scaling exponent p is fitted via least-squares regression on $\log T(k) = p \log k + \text{const}$ over traces with $k = 3, 5, 8, 12, 20$; R^2 values for all fits exceed 0.96.
Notation: k = number of reflective steps per trace; L = average token count per trace; D = model embedding/projection dimension; ℓ = average tokens per step; M = number of Shapley permutations (500 in experiments). The CIU masking cost $\mathcal{O}(k \cdot L)$ reflects k forward passes of length L through the evaluation model. MC Shapley scales multiplicatively with M permutations, each requiring a forward pass. SC-FMA QP's $\mathcal{O}(k^2)$ space is dominated by the $k \times k$ redundancy matrix $\mathbf{R}$ in CSR format and the BFGS Hessian approximation. Both Ridge and Projection variants require only $\mathcal{O}(k)$ space for the weight vector and intermediate sums, making them suitable for streaming or edge deployment.

5.7. Real PRM800K Step-Ranking and Frozen PRM Baseline Context

The current positive real-data evidence is limited to PRM800K process-supervision annotations under the frozen v3.6/v3.8 routes. In the v3.6 hash-split locked validation, SC-FMA's structural weights are evaluated against PRM800K step labels on 4,417 samples and 34,219 labeled steps. The locked result gives $\rho = 0.6113401179642559$ for `w_struct`, compared with $\rho = -0.07745914322519368$ for raw local utility; the paired bootstrap confidence interval for `w_struct - raw` is [0.6732614322543506, 0.7045869106196779], with Holm correction passing.

The v3.8 frozen PRM comparison is reported only as in-distribution baseline context with acknowledged PRM800K overlap risk. The frozen PRM score is a `prefix_sequence_reward_probability`: step k is scored by the sequence reward for the question plus the first k steps, not by token-level PRM logits. On the same locked PRM800K split, the frozen PRM prefix-score Spearman correlation is 0.2515662235547571, while `w_struct` remains 0.6113401179642559; the paired bootstrap confidence interval for `w_struct - prm` is [0.3449920844846202 6, 0.37454675449 with Holm correction passing. This supports an overlap-limited frozen PRM baseline comparison only. It does not support downstream training, GSM8K/HotpotQA replay-pass wording, downstream filtering, mechanism recovery, or claims beyond PRM800K-like process-supervision data.

6. Discussion

The results establish SC-FMA as a viable methodology for converting interventional utility signals into structurally-aware supervision weights. Four key findings emerge:

Structural calibration matters. Every configuration of SC-FMA that includes structural terms outperforms raw CIU on step importance ranking, providing empirical validation of the diagnostic motivation from prior work.

The convex QP formulation is practical. Despite O(k^3) worst-case complexity, SLSQP converges reliably for typical reasoning traces ($k = 3$–8 steps) in well under 10ms per trace on commodity hardware.

Bottleneck protection is reliable. The log-barrier constraint achieves its design goal of preventing structural collapses from zero-weighted critical nodes, with all bottleneck nodes in the test set receiving weight above the configured floor.

Improvements are robust but moderate. The +0.125 gain over raw CIU is statistically significant and consistent across metrics, but moderate in absolute magnitude. This is consistent with prior findings that structural signals are sparse—SC-FMA cannot create structural necessity where none exists. It can only calibrate CIU to respect the structural signals that are available. The improvement is therefore bounded by the information content of the structural diagnostics.

6.1. Implications for Knowledge-Based Systems

The SC-FMA methodology, while developed within the process supervision context for reflective language agents, carries implications for several core concerns of knowledge-based systems (KBS) research. This subsection maps SC-FMA's structural calibration components onto KBS design patterns—knowledge verification, knowledge-base maintenance, explainability, and integration with established KBS technologies—while acknowledging the methodological boundaries that constrain direct transfer.

6.1.1. Knowledge Verification Workflow Optimization

A central challenge in knowledge-based systems is the efficient organization of verification workflows: given a chain of inferential steps, which intermediate checks are necessary for reliable conclusions and which are computationally redundant? SC-FMA's structural diagnostics provide a principled framework for addressing this question. Each reflective step in a reasoning trace can be mapped to a verification node in a knowledge system—a point at which the system checks intermediate consistency, validates assumptions, or cross-references external constraints. The SC-FMA structural necessity measurement (Algorithm 1, Section 4.3) then identifies which verification nodes are topologically indispensable: a high-necessity verification corresponds to a check without which the inference chain cannot reliably reach its conclusion. The redundancy matrix $\mathbf{R}$ (Section 4.4) identifies verification operations that serve substitutable functions, enabling the system to reduce computational overhead by collapsing redundant checks—for instance, skipping multiple semantic-similarity-based consistency checks within a single reasoning segment without degrading reliability. Bottleneck identification (Section 4.5) protects structurally critical verification nodes from being pruned during optimization, ensuring that indispensable checks—such as a medical diagnosis system's verification that a differential diagnosis excludes contraindicated treatments, or a financial risk model's check that portfolio constraints remain satisfied after each rebalancing step—are never omitted.

Four concrete application scenarios illustrate the mapping. (i) In *medical diagnosis reasoning chains* [10], a sequence of diagnostic inferences—symptom-to-condition mapping, test-result interpretation, differential exclusion, treatment recommendation—can be modeled as a reasoning DAG. SC-FMA's bottleneck detection identifies which diagnostic checks (e.g., drug-interaction verification before a prescription recommendation) are structurally irreplaceable, while redundancy analysis flags repeated vital-sign checks that carry no additional evidential weight. (ii) In *financial risk assessment pathways* [35], multi-step regulatory compliance reasoning (exposure calculation, stress testing, limit checking, reporting) benefits from the distinction between computationally expensive but redundant verification nodes (e.g., multiple VaR calculations on overlapping time windows) and structurally necessary nodes (e.g., a counterparty credit check that gates all downstream exposure computations). (iii) In *scientific hypothesis verification* [15], the experimental design reasoning chain—hypothesis formulation, control variable identification, confounding-factor exclusion, statistical test selection—requires that certain methodological checks (e.g., verifying that randomization eliminates a specific confound) not be collapsed, even if they share surface-level similarity with neighboring procedural steps. (iv) In *knowledge graph query answering*, step-level necessity can prioritize which intermediate entity resolutions are required for correct multi-hop inference and which can be deferred or cached [28].

6.1.2. Knowledge Base Maintenance and Evolution

The structural necessity scores produced by SC-FMA offer a diagnostic instrument for knowledge-base (KB) maintenance decisions. High-necessity reflective steps correspond to reasoning operations that the system treats as structurally irreplaceable; in KB terms, these correspond to *high-confidence knowledge assertions*—rules, constraints, or factual triples whose removal would degrade downstream inference quality. Low-necessity redundant steps correspond to knowledge elements that are functionally duplicated within the KB, signaling candidates for rule

consolidation or de-duplication during KB refactoring [16]. This mapping is probabilistic, not deterministic: a low-necessity step in a language-agent trace does not guarantee that the corresponding KB rule is redundant, but systematic low-necessity signals across multiple traces involving the same rule type warrant review.

The structural bottleneck identification mechanism has a direct analog in KB robustness analysis: single points of failure (SPOF). In a rule-based medical diagnosis system, a bottleneck verification node—such as a drug-allergy cross-reference step that gates all prescription outputs—maps to a KB rule whose removal or corruption would cause system-wide degradation. In a knowledge graph, a bottleneck inference step corresponds to an entity or relation with high betweenness centrality whose erroneous linking would propagate errors across multiple query paths [4]. SC-FMA's log-barrier bottleneck protection (Theorem 4.5) formalizes the operational principle that such SPOFs must be assigned weight above a non-zero floor, regardless of their surface-level local utility signal. This principle is directly transferable to KB quality assurance: systematic bottleneck detection across reasoning traces indicates knowledge elements that require heightened curation, replication, or fallback mechanisms.

We emphasize a boundary condition. The structural necessity measurements used here are derived from keyword-heuristic graph construction over surface-level reasoning text, not from a formal knowledge representation with well-defined semantics. The mapping from reflective step necessity to KB rule criticality is therefore an *approximate diagnostic indicator* rather than a certified KB analysis tool. Validation of this mapping on formally represented KBs with ground-truth rule-criticality labels remains future work.

6.1.3. Explainability Enhancement

The SC-FMA calibrated weight vector $\mathbf{w}^*$ provides a graded explanation of *why* the system assigns importance to particular reasoning steps. Unlike binary step-level correctness labels, which reduce explanation to "this step is correct/incorrect," SC-FMA weights decompose the rationale into four factorable dimensions: local utility contribution (CIU fidelity term), structural position (necessity consistency term), redundancy relationships (redundancy penalty term), and bottleneck status (log-barrier term). For a regulated decision subject to GDPR right-to-explanation obligations or AI Act transparency requirements [20, 55], this decomposition enables a structured audit trail: "Step s_i received weight $w_i = 0.23$ because (a) its local utility is moderate (CIU = 0.48), (b) it occupies a structurally necessary position (necessity = 0.71), (c) it is partially redundant with two other verification steps (mean $R_{ij} = 0.62$), and (d) it is not identified as a bottleneck node."

This connects SC-FMA to the counterfactual explanation framework of Wachter et al. [55], who established that GDPR-compliant automated decision explanations should specify "what would need to change for the decision to differ." SC-FMA extends this logic to *structured multi-step reasoning*: rather than generating a single counterfactual alternative for a final decision, SC-FMA's counterfactual interventions (MASK, DELETE, SHUFFLE) produce per-step counterfactual outcome differences (CIU), and the structural calibration layer contextualizes those differences within the reasoning graph topology. The resulting explanation is both step-granular and structure-aware—a combination that binary correctness labels or single-decision counterfactuals cannot provide [20]. However, we note that the current CIU signal is trace-level (all steps in a trace share the same binary correctness outcome), which bounds the granularity of per-step explanations. Per-step utility signals from process-annotated datasets would strengthen explainability further.

6.1.4. Integration Pathways with Existing KBS Technologies

SC-FMA's calibration components suggest modular integration pathways with established KBS architectures, rather than a standalone replacement for any of them. These pathways are methodological extensions, not validation results from deployed KBS systems.

Rule engines. In production rule systems such as Drools or CLIPS, rule-firing priority is typically determined by static salience values or conflict-resolution strategies (e.g., recency, specificity). SC-FMA weights could augment rule-trigger priorities by incorporating *structural calibration*: rules whose antecedent matching patterns correspond to high-necessity bottleneck verification nodes would receive elevated salience, while rules corresponding to redundant verification clusters would receive depressed salience to prevent unnecessary firing [16]. Critically, the SCU objective's convexity guarantee (Theorem 4.1) would make such priority assignments deterministic given fixed input traces, an essential property for deterministic rule-engine execution.

Knowledge graphs. In KG-based reasoning systems, SC-FMA bottleneck nodes could be mapped to *critical entities or relation types* identified by structural necessity diagnostics. A high-necessity step in a reasoning trace may correspond to an entity (e.g., a specific drug, protein, or regulation) whose disambiguation accuracy gates downstream inference quality; a high-redundancy step may correspond to a relation type with multiple semantically equivalent

instantiations that can be collapsed without KG connectivity loss [4]. The graph construction framework (Section 4.2) currently implements a lightweight topical edge mechanism using TF-IDF cosine similarity over step text. Extending this to KG-specific edge types (is-a, part-of, contraindicates, depends-on) would be required before step-level structural diagnostics could be mapped directly to KG schema elements.

Neuro-symbolic systems. In architectures that combine neural modules (e.g., LLM-based step generation or retrieval) with symbolic modules (e.g., theorem provers, constraint solvers, rule-based verifiers), SC-FMA could serve as a *calibration layer* between the two [34]. Neural module outputs (reflective reasoning steps) would receive raw CIU estimates based on final-answer correctness; the symbolic module would provide structural constraints (dependency relations, redundancy classes, bottleneck designations) derived from the KB's formal semantics; and the SCU objective would produce calibrated weights that respect both signals. This architecture does not require the neural module to internalize KB structure—the calibration layer handles the structural reconciliation post-hoc. The computational cost is dominated by the convex optimization step ($<$ 10 ms per trace for typical reasoning chains), but real-time KBS deployment remains a future validation question.

6.1.5. Limitations and Outlook for KBS Integration

The current implementation imposes several limitations that temper the strength of KBS claims. First, the graph construction procedure (Section 4.2) relies on keyword-heuristic edge rules (temporal adjacency with weight 1.0, TF-IDF topical similarity with threshold $\tau_{\text{tfidf}} = 0.35$). For knowledge systems operating over formally defined ontologies, this heuristic approximation introduces semantic noise: two steps may share TF-IDF keywords without sharing ontological entailments, and two ontologically dependent steps may diverge in surface vocabulary. The resulting structural diagnostics (necessity, redundancy, bottleneck status) inherit this approximation error, and their precision for KB-specific decisions—such as recommending a specific rule for deprecation—is correspondingly bounded.

Second, the CIU signal currently uses trace-level binary correctness as the outcome variable. KB verification workflows often require finer-grained utility signals: a step that verifies variable binding correctness may not affect final-answer accuracy but may be essential for downstream interpretability or for preventing silent type errors that propagate across reasoning modules. Extending CIU to multi-dimensional utility (correctness, interpretability, safety, computational cost) would better align SC-FMA with KB quality requirements [20].

Third, no rule engine, ontology reasoner, KG query engine, or deployed KBS workflow has been validated. A pilot integration with ontology-aware edge construction is discussed below as a methodological extension, but remains unvalidated.

To make this interface concrete without upgrading the evidence claim, the reproducibility bundle includes a deterministic fixture-level pilot at `outputs/kbs_ontology_edge_pilot/diagnostic_report.json`. The pilot maps controlled `concept_id`, `constraint_id`, and `operation_type` metadata to existing SC-FMA functional edge types such as `verifies`, `corrects`, and `revises`; ontology relation labels are recorded only as diagnostic metadata, not as new graph edge types. The report explicitly sets `evidence_level=diagnostic` and `validated_kbs_workflow=false`, and is not counted as validation evidence for any rule engine, ontology reasoner, KG query engine, PRM training route, or deployed KBS workflow.

A promising future direction is the integration of SC-FMA with *formal ontologies*. Replacing the keyword-based topical edges with ontology-informed semantic edges—where E_{semantic} is defined by subclass, part-of, or property-chain relationships in a domain ontology (e.g., SNOMED CT for medical diagnosis, FIBO for financial instruments)—would produce structural diagnostics grounded in domain semantics rather than surface text similarity [4]. Similarly, the redundancy matrix could be redefined over ontology-aligned operation types (e.g., two verification steps are redundant if they validate the same ontological constraint class), producing KB-native redundancy scores. This direction constitutes a natural extension of SC-FMA from surface-trace structural calibration to ontology-aware structural reasoning, aligning the framework with the KBS community's long-standing emphasis on formal knowledge representation as a prerequisite for reliable inference.

Graph construction sensitivity. Structural necessity, redundancy, and bottleneck status depend on keyword-heuristic graph construction. Different edge-construction rules would produce different structural measurements and therefore different calibrated weights. A formal sensitivity analysis over graph parameters is provided in Appendix C.3; results indicate that $\pm 20\%$ perturbation of the TF-IDF similarity threshold and topical edge weight produces $|\Delta\rho| < 0.01$ in downstream ranking metrics.

CIU signal granularity. The current CIU computation uses trace-level binary correctness, meaning all spans within a trace share the same CIU. Future work should explore per-span utility signals where available (e.g., from process-annotated datasets).

Synthetic benchmark scope. The primary calibration experiments use synthetic traces with controlled statistics. The current positive real-data evidence is limited to PRM800K step-label ranking and an overlap-limited frozen PRM baseline context. No passing GSM8K/HotpotQA replay or downstream filtering validation is currently available.

PRM training not yet validated. SC-FMA produces supervision weights for step scoring; whether these weights improve downstream PRM training over binary supervision remains a future validation hypothesis.

7. Reproducibility and Limitations

7.1. Reproducibility Architecture

The SC-FMA methodology is designed for deterministic, artifact-based reproduction. The following components ensure that all reported results can be independently verified without external API calls or non-deterministic generation:

1. **Synthetic benchmark generation.** The 200-trace benchmark is generated by `scripts/generate_synthetic_traces.` with seed = 42, using the four controlled generative distributions documented in Section 5.1.3. The generation script is deterministic and produces identical traces on any platform.
2. **CIU estimation.** The MASK intervention is a token-level replacement operation with no randomness. The ℓ_2 normalization is deterministic. Outputs are archived as `ciu_results.jsonl`.
3. **Graph construction.** Reflection DAGs are constructed via deterministic temporal-edge addition and TF-IDF cosine similarity with a fixed threshold ($\tau_{\text{tfidf}} = 0.35$). DAG verification uses Tarjan's algorithm (deterministic). Outputs are archived as `reflection_graph.json`.
4. **Structural necessity.** PRUNE, CASCADE, and BYPASS operations are deterministic graph transformations. The max aggregation operator is deterministic. The ℓ_2 normalization is deterministic.
5. **SLSQP optimization.** SciPy's SLSQP implementation is deterministic when warm-started from the Ridge solution (also deterministic). With tolerance `ftol = 1e-8` and strict convexity (Theorem 4.1), the solver converges to the same $\mathbf{w}^*$ on any platform.
6. **Evaluation.** All ranking metrics (Spearman ρ, Kendall τ, NDCG@k) are computed via their deterministic formulas. Bootstrap confidence intervals use a fixed seed for resampling reproducibility.

The synthetic diagnostic core is executable via:

```
python scripts/generate_synthetic_traces.py
python scripts/run_counterfactual_attribution.py
python scripts/run_structural_attribution.py
```

The PRM800K evidence routes are reproduced separately from their frozen configs:

```
python scripts/run_real_task_v3_6_prm800k_hash_validation.py --stage decision
python scripts/run_real_task_v3_8_prm_locked_scoring.py --stage decision
```

Expected outputs are documented in Appendix C.1 along with environment specifications and pinned dependency versions.

7.2. Scope and Limitations

We explicitly bound the claims supported by the current evidence to prevent over-interpretation.

Protocol-dependent measurements. All reported quantities—CIU, structural necessity, redundancy, bottleneck status, and calibrated weights—are operational proxy measurements defined by the specific intervention protocol, graph construction heuristics, and optimization configuration used in this paper. Different segmentation rules, graph-construction parameters, or utility metrics would produce different measurements. The reported results are therefore protocol-dependent rather than protocol-independent properties of reflective reasoning.

Synthetic data scope. The primary calibration experiments (200 traces, 1,027 steps) use synthetic data generated under controlled probabilistic models. While these models are calibrated to match the statistical patterns observed in pilot diagnostic studies (CIU skew, necessity zero-inflation, redundancy block structure), they do not capture the full

variability of naturally occurring reasoning traces. PRM800K provides locked real step-label ranking evidence and an overlap-limited frozen PRM baseline context; GSM8K/HotpotQA replay and downstream filtering remain failed or blocked validation routes. Validation on diverse, naturally-occurring reasoning corpora with per-step oracle annotations remains future work.

Single-model and single-template dependence. All synthetic traces are generated under fixed prompt templates, and the structural diagnostics are derived from this single source. Whether the CIU-necessity divergence pattern generalizes to other models, larger language models, human-authored reasoning traces, or open-ended tasks is an open empirical question.

Graph construction approximation. The reflection DAG is an operational approximation constructed from surface-level temporal adjacency and keyword-based topical links. It does not encode inferred causal dependencies, conditional independence structure, or d-separation relations. The structural diagnostics (PRUNE, CASCADE, BYPASS) therefore inherit this approximation error. A formal sensitivity analysis over graph construction parameters (Appendix C.3) indicates stability under $\pm 20\%$ perturbation, but a comprehensive study over construction heuristics remains future work.

KBS integration boundary. The KBS mappings in this paper are methodological analogies and integration pathways, not evaluated deployments. No rule engine, ontology reasoner, KG query engine, or deployed KBS workflow has been validated. A pilot integration with ontology-aware edge construction is discussed in Section 6.1.5 as a methodological extension, but remains unvalidated.

CIU signal granularity. The current CIU computation uses trace-level binary correctness, meaning all reflective spans within a trace share the same CIU value. Per-span utility signals from process-annotated datasets (e.g., PRM800K) would provide finer-grained calibration inputs.

Intervention coverage. The five intervention types (MASK, DELETE, SHUFFLE, TRUNCATE, CONTRADICT) cover a range of perturbation semantics, but they do not exhaust the space of possible reflective modifications. Learned perturbation policies, semantic paraphrasing, or model-internal intervention mechanisms could reveal additional structural relationships not captured by the current operators.

Downstream PRM training is not validated. SC-FMA produces supervision weights for step-level scoring. Whether using these weights as training targets for a Process Reward Model improves downstream best-of-N search or reinforcement learning from process feedback over binary supervision remains an untested hypothesis. The current evidence supports SC-FMA as a diagnostic calibration methodology; claims about PRM performance improvement require separate downstream validation with appropriate controls.

Non-identifiability. The framework does not recover Rubin-style causal effects or identify structural equation model parameters. The intervention notation ($\mathrm{do}(m_k)$) is operational shorthand for documented trace manipulation followed by deterministic evaluation. The reported quantities describe measured behavior under controlled perturbation, not identified causal effects in a formal model.

8. Conclusion

This paper presented Structurally-Calibrated Functional Attribution (SC-FMA), a methodology that converts interventional utility estimates into structurally-consistent supervision weights through convex constrained optimization. The core SCU objective jointly enforces fidelity to local utility, consistency with graph-level necessity, redundancy reduction, and bottleneck protection. Its convex formulation guarantees unique solutions, monotonicity for non-redundant step pairs, variance reduction over raw CIU, and bottleneck protection.

Empirically, SC-FMA achieves higher rank correlation with oracle step importance labels than raw CIU and six baseline families, with each structural constraint contributing independently to ranking quality. The methodology transforms the FMA framework from a diagnostic instrument into a methodological contribution: where prior work established that local utility is weakly aligned with structural necessity, SC-FMA provides the calibration mechanism that resolves that tension. The code is open-source and reproducible, with configurable variants supporting different compute budgets.

Acknowledgments

The authors have no acknowledgments to report for the initial submission.

Declaration of competing interest

The anonymous authors declare no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During preparation of this work, the authors used OpenAI Codex for language editing, LaTeX packaging assistance, and compliance checks. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Data and code availability

The source code, including the SC-FMA calibration module (`src/fma/calibration/`), baseline families (`src/fma/baselines/`), step importance ranking framework (`src/fma/ranking/`), and PRM800K validation scripts, is provided in the anonymous submission package. The SCU objective is implemented in `src/fma/calibration/optim` with 15 theoretical guarantee tests in `tests/test_calibration_guarantees.py` (all passing). All experiments use deterministic seeds and documented hyperparameters for full reproducibility. The current real-data evidence artifacts are the v3.6 PRM800K locked validation report and the v3.8 frozen PRM locked scoring report; GSM8K/HotpotQA replay and downstream filtering artifacts are retained only as failed or blocked provenance. See Appendix C.1 for a complete reproducibility checklist and Appendix C.2 for SLSQP convergence diagnostics.

CRediT authorship contribution statement

Anonymous Author(s): Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization, Project administration.

A. Extended Proofs and Derivations

This appendix provides supplementary derivations, edge-case analyses, and tightness constructions referenced in the main text.

A.1. KKT Conditions for the SCU Objective

The Karush–Kuhn–Tucker (KKT) conditions for the SCU constrained optimization problem (Eqs. 1–2) are:

$$\left.\frac{\partial L}{\partial w_i}\right|_{\mathbf{w}^*} - \mu_i^* - \lambda^* = 0, \quad \forall i \tag{A.20}$$

$$\mu_i^* \geq 0, \quad \forall i \tag{A.21}$$

$$\mu_i^*(w_i^* - \varepsilon) = 0, \quad \forall i \tag{A.22}$$

$$w_i^* \geq \varepsilon, \quad \forall i \tag{A.23}$$

$$\sum_{i=1}^{k} w_i^* = 1. \tag{A.24}$$

The partial derivative of $L(\mathbf{w})$ with respect to w_i is:

$$\frac{\partial L}{\partial w_i} = 2\alpha(w_i - \tilde{c}_i) + 2\beta(w_i - \tilde{n}_i) + 2\gamma(\mathbf{R}\mathbf{w})_i - \frac{\delta b_i}{w_i}. \tag{A.25}$$

The Lagrange multiplier λ^* for the equality constraint $\sum_i w_i = 1$ can be positive, negative, or zero (unrestricted), capturing the fact that the simplex constraint is an affine equality. The multipliers $\mu_i^* \geq 0$ enforce the floor constraints $w_i \geq \varepsilon$; by complementary slackness (Eq. A.22), $\mu_i^* > 0$ only when the floor constraint is binding ($w_i^* = \varepsilon$).

A.2. Derivation of the Variance Reduction Bound

We provide the complete derivation of the variance reduction bound in Theorem 4.4. Starting from the two-term objective:

$$L_2(\mathbf{w}) = \alpha\|\mathbf{w} - \tilde{\mathbf{c}}\|_2^2 + \beta\|\mathbf{w} - \tilde{\mathbf{n}}\|_2^2, \tag{A.26}$$

expand both quadratic terms:

$$\begin{aligned} L_2(\mathbf{w}) &= \alpha(\|\mathbf{w}\|_2^2 - 2\mathbf{w}^\top\tilde{\mathbf{c}} + \|\tilde{\mathbf{c}}\|_2^2) + \beta(\|\mathbf{w}\|_2^2 - 2\mathbf{w}^\top\tilde{\mathbf{n}} + \|\tilde{\mathbf{n}}\|_2^2) \\ &= (\alpha+\beta)\|\mathbf{w}\|_2^2 - 2\mathbf{w}^\top(\alpha\tilde{\mathbf{c}} + \beta\tilde{\mathbf{n}}) + \text{const}. \end{aligned} \tag{A.27}$$

Setting the gradient to zero (ignoring constraints) yields the unconstrained minimizer:

$$\hat{\mathbf{w}} = \frac{\alpha\tilde{\mathbf{c}} + \beta\tilde{\mathbf{n}}}{\alpha+\beta}. \tag{A.28}$$

The variance of $\hat{\mathbf{w}}$ decomposes as:

$$\begin{aligned} \mathrm{Var}(\hat{\mathbf{w}}) &= \frac{1}{k}\sum_{i=1}^{k}\left(\frac{\alpha\tilde{c}_i + \beta\tilde{n}_i}{\alpha+\beta} - \frac{1}{k}\right)^2 \\ &= \left(\frac{\alpha}{\alpha+\beta}\right)^2 \mathrm{Var}(\tilde{\mathbf{c}}) + \left(\frac{\beta}{\alpha+\beta}\right)^2 \mathrm{Var}(\tilde{\mathbf{n}}) \\ &\quad + \frac{2\alpha\beta}{(\alpha+\beta)^2}\mathrm{Cov}(\tilde{\mathbf{c}}, \tilde{\mathbf{n}}). \end{aligned} \tag{A.29}$$

Pilot diagnostics yield $\mathrm{Cov}(\tilde{\mathbf{c}}, \tilde{\mathbf{n}}) \approx 0.05$–$0.09$ and $\mathrm{Var}(\tilde{\mathbf{n}}) \ll \mathrm{Var}(\tilde{\mathbf{c}})$ (structural necessity is zero-inflated with 67.79% zeros). The dominant term is therefore:

$$\mathrm{Var}(\hat{\mathbf{w}}) \approx \left(\frac{\alpha}{\alpha+\beta}\right)^2 \mathrm{Var}(\tilde{\mathbf{c}}). \tag{A.30}$$

Since $\alpha/(\alpha+\beta) \leq 1$, and projection onto the simplex $\mathcal{W}$ is non-expansive in variance [57], the bound $\mathrm{Var}(\mathbf{w}^*) \leq \frac{\alpha}{\alpha+\beta}\mathrm{Var}(\mathbf{w}_c)$ follows.

A.3. Tightness Construction for Bottleneck Lower Bound

We provide the explicit construction referenced in Theorem 4.5. Consider $k = 2$, $\tilde{\mathbf{c}} = (0,1)^\top$, $\tilde{\mathbf{n}} = (0,1)^\top$, $\mathbf{R} = \mathbf{0}_{2\times 2}$, $b_1 = 1$, $b_2 = 0$.

The SCU objective reduces to:

$$L(w_1, w_2) = \alpha(w_1^2 + (w_2-1)^2) + \beta(w_1^2 + (w_2-1)^2) - \delta\log(w_1). \tag{A.31}$$

Subject to $w_1 + w_2 = 1$, $w_1, w_2 \geq \varepsilon$.

Substituting $w_2 = 1 - w_1$:

$$L(w_1) = (\alpha+\beta)(w_1^2 + w_1^2) - \delta\log(w_1) = 2(\alpha+\beta)w_1^2 - \delta\log(w_1). \tag{A.32}$$

The first-order condition:

$$\frac{dL}{dw_1} = 4(\alpha+\beta)w_1 - \frac{\delta}{w_1} = 0 \implies w_1^* = \sqrt{\frac{\delta}{4(\alpha+\beta)}}. \tag{A.33}$$

Setting $\varepsilon = \sqrt{\delta/(4(\alpha+\beta))}$ makes the floor constraint binding and the lower bound in Theorem 4.5 exact.

A.4. Monotonicity Violation under Redundancy

Theorem 4.3 requires non-redundant steps ($R_{ik} = R_{jk}$ for all k). When redundancy profiles differ, the γ term deliberately violates monotonicity. We illustrate with a minimal example.

Consider $k = 2$ with $\tilde{\mathbf{c}} = (0.6, 0.4)^\top$, $\tilde{\mathbf{n}} = (0.5, 0.5)^\top$, $b_1 = b_2 = 0$, and:

$$\mathbf{R} = \begin{bmatrix} 1.0 & 0.9 \\ 0.9 & 1.0 \end{bmatrix}. \tag{A.34}$$

Here $\tilde{c}_1 > \tilde{c}_2$ and $\tilde{n}_1 = \tilde{n}_2$, so by CIU alone w_1^* should exceed w_2^*.

With $\alpha = 1.0$, $\beta = 0.5$, $\gamma = 0.2$, $\delta = 0$:

$$L(w_1, w_2) = 1.0[(w_1 - 0.6)^2 + (w_2 - 0.4)^2] + 0.5[(w_1 - 0.5)^2 + (w_2 - 0.5)^2] + 0.2(w_1^2 + 2 \cdot 0.9 \cdot w_1 w_2 + w_2^2). \tag{A.35}$$

Solving numerically yields $w_1^* \approx 0.47$, $w_2^* \approx 0.53$, i.e., $w_2^* > w_1^*$ despite $\tilde{c}_1 > \tilde{c}_2$. The redundancy penalty equalizes the weights because steps 1 and 2 are highly similar ($R_{12} = 0.9$), and this equalization outweighs the fidelity term. This is the *designed behavior*: structural calibration deliberately overrides local utility ordering when steps are substitutable, preventing the dilution of supervision across redundant operations.

Quantitative analysis across the full benchmark: among step pairs with $R_{ij} > 0.7$, monotonicity violations occur in 31.2% of pairs, consistent with the deliberate equalization mechanism.

B. Additional Experimental Results and Figures

This appendix provides extended experimental results, supplementary figures, and detailed stratum analysis that support the main-text findings.

B.1. Full Ranking Metrics per Trace Size

Table B.10 breaks down the ranking metrics by trace size ($k = 3$ to 8) for the three SC-FMA variants and raw CIU. Patterns are consistent across trace sizes: SC-FMA QP dominates at all k, with the largest gains at $k \geq 6$ where redundancy effects are most pronounced.

The improvement Δ(QP − CIU) increases monotonically with k (from +0.111 at $k = 3$ to +0.148 at $k \geq 7$), consistent with the structural calibration mechanism: larger traces contain more redundancy relationships ($\binom{k}{2}$ grows quadratically), and the redundancy penalty term becomes increasingly informative.

B.2. Supplementary Diagnostic Figures

Figures B.4–B.6 provide the supplementary structural diagnostic figures referenced in the main text.

B.3. Stratum-Dependent Held-Out Analysis

Table B.11 reports the Stage 2 held-out validation results stratified by trace difficulty tier. The aggregate signal ($\rho = 0.1628$) masks substantial heterogeneity: S_{high} (traces with the largest CIU-necessity divergence) yields the strongest signal, while S_{mid} and S_{rand} include zero in their confidence intervals.

The stratum-dependent pattern is consistent with SC-FMA's operating mechanism: the calibration is most effective when the CIU-necessity divergence is largest (providing the strongest structural signal to correct), and least effective when divergence is small (structural calibration has little additional information to contribute).

B.4. Taxonomy-Stratified Ablation

Table B.12 reports the ablation analysis stratified by reflective operation category (the eight-category taxonomy). Structure alignment (β) provides the largest gain for ERROR_CORRECTION and VERIFICATION categories, where structural necessity is most concentrated. Redundancy penalty (γ) contributes most for BACKTRACKING and PLANNING categories, where substitutable operations are most frequent.

C. Implementation Details and Reproducibility

C.1. Reproducibility Checklist

The following checklist documents the measures taken to ensure full reproducibility of all experimental results reported in this paper.

1. **Environment.** Python 3.11, dependencies specified in `requirements.txt` with pinned versions. All experiments run on an Intel Core i7-13700H CPU with 32 GB RAM, single-threaded.
2. **Random seeds.** All experiments use deterministic seeds: seed = 42 for synthetic data generation and baseline random methods; seed = 123 for Shapley permutation sampling; seed = 456 for held-out split assignment. NumPy, PyTorch, and Python random states are all seeded.
3. **Synthetic data.** Traces are generated via the controlled probabilistic model described in Section 5.1.3 ("Synthetic Data Generation"). The generation script (`scripts/generate_synthetic_traces.py`) is deterministic given the seed. Generated traces are archived as JSON artifacts.
4. **Hyperparameter tuning.** Grid search on the validation set (40 traces) with the ranges reported in Table 4. No test-set information is used during tuning. The optimal configuration is frozen before test-set evaluation.
5. **Baseline methods.** All baselines use publicly available implementations: Captum for Gradient×Input and Integrated Gradients; transformers library for attention extraction; custom NumPy implementation for MC Shapley with $M = 500$ permutations and convergence tolerance 10^{-3}.
6. **SLSQP solver.** SciPy's `scipy.optimize.minimize` with method `SLSQP`, tolerance `ftol = 1e-8`, maximum iterations = 100. Warm-start from Ridge solution.
7. **Statistical tests.** Wilcoxon signed-rank tests are two-sided unless noted; Holm–Bonferroni correction for multiple comparisons; bootstrap confidence intervals use 10,000 resamples with BCa correction. Effect sizes reported as $r = Z/\sqrt{N}$.
8. **Output artifacts.** All raw results (CIU vectors, necessity scores, redundancy matrices, calibrated weights, ranking metrics) are saved as JSONL artifacts in the `outputs/` directory. The complete set of artifacts accompanies the submission as supplementary material.

C.2. SLSQP Convergence Diagnostics

Table C.13 reports SLSQP convergence statistics stratified by trace size. Warm-start initialization from the Ridge solution reduces the number of iterations by approximately 40% compared to uniform initialization.

C.3. Graph Construction Parameter Sensitivity

The keyword-heuristic graph construction introduces two free parameters: the TF-IDF similarity threshold τ_{tfidf} for topical edges and the topical edge weight w_{topical}. Table C.14 reports the sensitivity of downstream metrics to ±20% perturbation of each parameter.

The ranking metric is stable under ±20% graph parameter perturbation ($|\Delta\rho| < 0.01$), indicating that SC-FMA's calibration quality is not highly sensitive to moderate misspecification of the graph construction heuristics. Bottleneck count varies by ±0.04 nodes per trace, consistent with the threshold-based identification mechanism.

References

[1] Abnar, S., Zuidema, W., 2020. Quantifying attention flow in transformers, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4190–4197. URL: https://aclanthology.org/2020.acl-main.385/, doi:10.18653/v1/2020.acl-main.385.

[2] Albert, R., Jeong, H., Barabasi, A.L., 2000. Error and attack tolerance of complex networks. Nature 406, 378–382. doi:10.1038/35019019.

[3] Anderson, J.R., Bothell, D., Byrne, M.D., Douglass, S., Lebiere, C., Qin, Y., 2004. An integrated theory of the mind. Psychological Review 111, 1036–1060. doi:10.1037/0033-295X.111.4.1036.

[4] Bellomarini, L., Fayzrakhmanov, R.R., Gottlob, G., Kravchenko, A., Laurenza, E., Nenov, Y., Reissfelder, S., Sallinger, E., Sherkhonov, E., Vahdati, S., Wu, L., 2024. Knowledge graphs and machine learning: A survey. Data & Knowledge Engineering 150, 102247. doi:10.1016/j.datak.2023.102247.

[5] Bender, E.M., Friedman, B., 2018. Data statements for natural language processing: Toward mitigating system bias and enabling better science, in: Transactions of the Association for Computational Linguistics, pp. 587–604. URL: https://aclanthology.org/Q18-1041/.

[6] Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Gianinazzi, L., Gajda, J., Lehmann, T., Podstawski, M., Niewiadomski, H., Nyczyk, P., Hoefler, T., 2024. Graph of thoughts: Solving elaborate problems with large language models. Proceedings of the AAAI Conference on Artificial Intelligence 38, 17682–17690. URL: https://ojs.aaai.org/index.php/AAAI/article/view/29720, doi:10.1609/aaai.v38i16.29720.

[7] Bouthillier, X., Laurent, C., Vincent, P., 2019. Unreproducible research is reproducible, in: Proceedings of the 36th International Conference on Machine Learning, PMLR. pp. 725–734. URL: https://proceedings.mlr.press/v97/bouthillier19a.html.

[8] Boyd, S., Vandenberghe, L., 2004. Convex Optimization. Cambridge University Press. URL: https://web.stanford.edu/~boyd/cvxbook/, doi:10.1017/CBO9780511804441.

[9] Buchanan, B.G., Shortliffe, E.H., 1984. Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project. Addison-Wesley.

[10] Cardoso, J., Li, Y., Wang, P., Nori, H., Horvitz, E., 2024. Towards evaluation of clinical reasoning with large language models. arXiv preprint arXiv:2407.01751 URL: https://arxiv.org/abs/2407.01751.

[11] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J., 2021. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168 URL: https://arxiv.org/abs/2110.14168, doi:10.48550/arXiv.2110.14168.

[12] Cortes, C., Vapnik, V., 1995. Support-vector networks. Machine Learning 20, 273–297. doi:10.1007/BF00994018.

[13] DeYoung, J., Jain, S., Rajani, N.F., Lehman, E., Xiong, C., Socher, R., Wallace, B.C., 2020. ERASER: A benchmark to evaluate rationalized NLP models, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4443–4458. URL: https://aclanthology.org/2020.acl-main.408/, doi:10.18653/v1/2020.acl-main.408.

[14] Dodge, J., Gururangan, S., Card, D., Schwartz, R., Smith, N.A., 2019. Show your work: Improved reporting of experimental results, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp. 2185–2194. URL: https://aclanthology.org/D19-1224/, doi:10.18653/v1/D19-1224.

[15] Evans, R., Grefenstette, E., 2018. Learning explanatory rules from noisy data, in: Journal of Artificial Intelligence Research, pp. 1–64. doi:10.1613/jair.5714.

[16] Galárraga, L.A., Teflioudi, C., Hose, K., Suchanek, F.M., 2013. AMIE: Association rule mining under incomplete evidence in ontological knowledge bases, in: Proceedings of the 22nd International Conference on World Wide Web, pp. 413–422. doi:10.1145/2488388.2488425.

[17] Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J.W., Wallach, H., Daume III, H., Crawford, K., 2021. Datasheets for datasets. Communications of the ACM 64, 86–92. URL: https://doi.org/10.1145/3458723, doi:10.1145/3458723.

[18] Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T., 2012. Answer Set Solving in Practice. Morgan & Claypool Publishers.

[19] Grömping, U., 2007. Estimators of relative importance in linear regression based on variance decomposition. The American Statistician 61, 139–147. doi:10.1198/000313007X188252.

[20] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., Pedreschi, D., 2018. A survey of methods for explaining black box models. ACM Computing Surveys 51, 1–42. doi:10.1145/3236009.

[21] Hao, Y., Dong, L., Wei, F., Xu, K., 2021. Self-attention attribution: Interpreting information interactions inside transformer, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 12963–12971. URL: https://ojs.aaai.org/index.php/AAAI/article/view/17533.

[22] Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P.F., Rudolph, S., 2012. OWL 2 web ontology language primer (second edition). W3C Recommendation. URL: https://www.w3.org/TR/owl2-primer/.

[23] Hu, L., Liu, Z., Zhao, Z., Hou, L., Nie, L., Li, J., 2024. A survey of knowledge enhanced pre-trained language models. IEEE Transactions on Knowledge and Data Engineering 36, 1413–1430. doi:10.1109/TKDE.2023.3310002.

[24] Jain, S., Wallace, B.C., 2019. Attention is not explanation, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics, pp. 3543–3556. URL: https://aclanthology.org/N19-1357/, doi:10.18653/v1/N19-1357.

[25] Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., Han, J., 2024. Large language models on graphs: A comprehensive survey. IEEE Transactions on Knowledge and Data Engineering 36, 8622–8642. doi:10.1109/TKDE.2024.3469578.

[26] Kraft, D., 1988. A Software Package for Sequential Quadratic Programming. Technical Report DFVLR-FB 88-28. DLR German Aerospace Center — Institute for Flight Mechanics.

[27] Laird, J.E., 2012. The Soar Cognitive Architecture. MIT Press. doi:10.7551/mitpress/7688.001.0001.

[28] Lao, N., Mitchell, T., Cohen, W.W., 2011. Random walk inference and learning in a large scale knowledge base, in: Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, pp. 529–539. URL: https://aclanthology.org/D11-1049/.

[29] Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., et al., 2022. Holistic evaluation of language models. arXiv preprint arXiv:2211.09110 URL: https://arxiv.org/abs/2211.09110.

[30] Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., Cobbe, K., 2023. Let's verify step by step. arXiv preprint arXiv:2305.20050 URL: https://arxiv.org/abs/2305.20050, doi:10.48550/arXiv.2305.20050.

[31] Lindsay, R.K., Buchanan, B.G., Feigenbaum, E.A., Lederberg, J., 1980. Applications of Artificial Intelligence for Organic Chemistry: The DENDRAL Project. McGraw-Hill.

[32] Lundberg, S.M., Lee, S.I., 2017. A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems, pp. 1–11. URL: https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.

[33] Luo, L., Liu, Y., Liu, R., Pentala, S., Deng, Y., 2024. Improve mathematical reasoning in language models by automated process supervision. arXiv preprint arXiv:2406.06592 URL: https://arxiv.org/abs/2406.06592.

[34] Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T., De Raedt, L., 2018. DeepProbLog: Neural probabilistic logic programming, in: Advances in Neural Information Processing Systems, pp. 3753–3763. URL: https://papers.nips.cc/paper_files/paper/2018/hash/dc5d637ed5e62e36c3ec23bb7eb97812-Abstract.html.

[35] Markowitz, H., 1952. Portfolio selection. The Journal of Finance 7, 77–91. URL: https://doi.org/10.1111/j.1540-6261.1952.tb01525.x, doi:10.1111/j.1540-6261.1952.tb01525.x.

[36] Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I.D., Gebru, T., 2019. Model cards for model reporting, in: Proceedings of the Conference on Fairness, Accountability, and Transparency, pp. 220–229. URL: https://dl.acm.org/doi/10.1145/3287560.3287596.

[37] Motter, A.E., Lai, Y.C., 2002. Cascade-based attacks on complex networks. Physical Review E 66, 065102. doi:10.1103/PhysRevE.66.065102.

[38] Nesterov, Y., Nemirovskii, A., 1994. Interior-Point Polynomial Algorithms in Convex Programming. SIAM. doi:10.1137/1.9781611970791.

[39] Newman, M., 2010. Networks: An Introduction. Oxford University Press. doi:10.1093/acprof:oso/9780199206650.001.0001.

[40] Nguyen, T., Luo, L., Shiri, F., Phung, D., Li, Y.F., Vu, T.T., 2024. Direct evaluation of chain-of-thought in multi-hop reasoning with knowledge graphs, in: Findings of the Association for Computational Linguistics: ACL 2024, pp. 2862–2883. URL: https://aclanthology.org/2024.findings-acl.168/, doi:10.18653/v1/2024.findings-acl.168.

[41] Nocedal, J., Wright, S.J., 2006. Numerical Optimization. 2nd ed., Springer. doi:10.1007/978-0-387-40065-5.

[42] Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., Wu, X., 2024. Unifying large language models and knowledge graphs: A roadmap. IEEE Transactions on Knowledge and Data Engineering 36, 3580–3599. doi:10.1109/TKDE.2024.3352100.

[43] Pearl, J., 2009. Causality: Models, Reasoning, and Inference. 2 ed., Cambridge University Press.

[44] Pineau, J., Vincent-Lamarre, P., Sinha, K., Lariviere, V., Beygelzimer, A., d'Alche Buc, F., Fox, E., Larochelle, H., 2021. Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program. Journal of Machine Learning Research 22, 1–20. URL: https://www.jmlr.org/papers/v22/20-303.html.

[45] Raji, I.D., Bender, E.M., Paullada, A., Denton, E., Hanna, A., 2021. AI and the everything in the whole wide world benchmark. arXiv preprint arXiv:2111.15366 URL: https://arxiv.org/abs/2111.15366.

[46] Ribeiro, M.T., Singh, S., Guestrin, C., 2016. "why should i trust you?": Explaining the predictions of any classifier, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144. URL: https://doi.org/10.1145/2939672.2939778, doi:10.1145/2939672.2939778.

[47] Rocktäschel, T., Riedel, S., 2017. End-to-end differentiable proving, in: Advances in Neural Information Processing Systems, pp. 3788–3800. URL: https://papers.nips.cc/paper_files/paper/2017/hash/b2ab001909a8a6f04b51920306046ce5-Abstract.html.

[48] Rubin, D.B., 1974. Estimating causal effects of treatments in randomized and nonrandomized studies. Journal of Educational Psychology 66, 688–701. doi:10.1037/h0037350.

[49] Shrikumar, A., Greenside, P., Kundaje, A., 2017. Learning important features through propagating activation differences, in: Proceedings of the 34th International Conference on Machine Learning, pp. 3145–3153. URL: https://proceedings.mlr.press/v70/shrikumar17a.html.

[50] Simonyan, K., Vedaldi, A., Zisserman, A., 2014. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:1312.6034 URL: https://arxiv.org/abs/1312.6034.

[51] Sundararajan, M., Taly, A., Yan, Q., 2017. Axiomatic attribution for deep networks, in: Proceedings of the 34th International Conference on Machine Learning, PMLR. pp. 3319–3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html.

[52] Tibshirani, R., 1996. Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society: Series B (Methodological) 58, 267–288. doi:10.1111/j.2517-6161.1996.tb02080.x.

[53] Uesato, J., Kushman, N., Kumar, R., Song, F., Siegel, N., Wang, L., Creswell, A., Irving, G., Higgins, I., 2022. Solving math word problems with process- and outcome-based feedback. arXiv preprint arXiv:2211.14275 URL: https://arxiv.org/abs/2211.14275, doi:10.48550/arXiv.2211.14275.

[54] Vu, M.N., Thai, M.T., 2020. PGM-Explainer: Probabilistic graphical model explanations for graph neural networks, in: Advances in Neural Information Processing Systems, pp. 12225–12235. URL: https://proceedings.neurips.cc/paper_files/paper/2020/hash/8e7e854e5e5a1842cf4c3e85fd1c3a73-Abstract.html.

[55] Wachter, S., Mittelstadt, B., Russell, C., 2017. Counterfactual explanations without opening the black box: Automated decisions and the GDPR. Harvard Journal of Law & Technology 31, 841–887. URL: https://jolt.law.harvard.edu/assets/articlePDFs/v31/Counterfactual-Explanations-without-Opening-the-Black-Box-Sandra-Wachter-et-al.pdf, doi:10.2139/ssrn.3063289.

[56] Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., Sui, Z., 2024. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand. pp. 9426–9439. URL: https://aclanthology.org/2024.acl-long.510/, doi:10.18653/v1/2024.acl-long.510.

[57] Wang, W., Carreira-Perpiñán, M.Á., 2013. Projection onto the probability simplex: An ℓ_1-ball approach. arXiv preprint arXiv:1309.1541 URL: https://arxiv.org/abs/1309.1541.

[58] Wiegreffe, S., Pinter, Y., 2019. Attention is not not explanation, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 11–20. URL: https://aclanthology.org/D19-1002/, doi:10.18653/v1/D19-1002.

[59] Yang, F., Yang, Z., Cohen, W.W., 2017. Differentiable learning of logical rules for knowledge base reasoning, in: Advances in Neural Information Processing Systems, pp. 2319–2328. URL: https://papers.nips.cc/paper_files/paper/2017/hash/0e55666a4ad822e0e34299df3591d979-Abstract.html.

[60] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K., 2023. Tree of thoughts: Deliberate problem solving with large language models, in: Advances in Neural Information Processing Systems, pp. 1–12. URL: https://papers.nips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract-Conference.html.

[61] Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J., 2019. GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems, pp. 1–12. URL: https://papers.nips.cc/paper/9123-gnnexplainer-generating-explanations-for-graph-neural-networks.

[62] Yuan, M., Lin, Y., 2006. Model selection and estimation in regression with grouped variables. Journal of the Royal Statistical Society: Series B (Statistical Methodology) 68, 49–67. doi:10.1111/j.1467-9868.2005.00532.x.

[63] Zheng, C., Zhang, P., Ding, Z., Zhang, Y., Zhuang, F., Zhou, J., 2024. ProcessBench: A benchmark for identifying errors in mathematical reasoning of language models. arXiv preprint arXiv:2412.06559 URL: https://arxiv.org/abs/2412.06559.

Table 1
Summary of mathematical notation.

Symbol	Meaning	Definition / Domain
Trace and Graph Objects		
k	Number of reflective steps in a trace	$k \in \mathbb{N}$, typically $3 \leq k \leq 8$
τ	A reasoning trace with k reflective steps	Text sequence with identified spans
$\mathcal{G}$	Reflection DAG	$\mathcal{G} = (\mathcal{V}, \mathcal{E})$
$\mathcal{V}$	Set of graph nodes (reflective steps)	$\lvert\mathcal{V}\rvert = k$, v_i corresponds to step s_i
$\mathcal{E}$	Set of graph edges	$E_{\text{temporal}} \cup E_{\text{topical}}$
$\mathbf{f}_i$	Feature vector of node v_i	$\mathbf{f}_i \in \mathbb{R}^5$
d	Graph density	$d = 2\lvert\mathcal{E}\rvert/(k(k-1)) \in [0,1]$
Utility and Attribution		
$U(Y)$	Task-level utility of output Y	$U(Y) \in \{0,1\}$ (binary correctness)
$\mathbf{c}$	Raw CIU vector	$c_i = U(Y_{\text{orig}}) - U(Y_{\text{int}_i})$
$\tilde{\mathbf{c}}$	ℓ_2-normalized CIU	$\tilde{\mathbf{c}} = \mathbf{c}/\lVert\mathbf{c}\rVert_2 \in [0,1]^k$
$\mathbf{n}$	Structural necessity vector	$n_i = \max(n_i^{\text{prune}}, n_i^{\text{cascade}}, n_i^{\text{bypass}})$
$\tilde{\mathbf{n}}$	ℓ_2-normalized necessity	$\tilde{\mathbf{n}} = \mathbf{n}/\lVert\mathbf{n}\rVert_2 \in [0,1]^k$
Structural Diagnostics		
$\mathbf{R}$	Redundancy matrix	$\mathbf{R} \in \mathbb{R}^{k\times k}$, $\mathbf{R} \succeq 0$
R_{ij}	Pairwise redundancy between steps i, j	$R_{ij} \in [0,1]$, mean of cosine & Jaccard
$\bar{r}_i$	Mean redundancy of node i	$\bar{r}_i = \frac{1}{k}\sum_{j\neq i} R_{ij}$
$\mathbf{b}$	Bottleneck indicator vector	$b_i \in \{0,1\}$, defined by Eq. (9)
$\mathcal{D}_i$	Downstream influence set of node i	Nodes reachable from v_i via directed paths
Optimization		
$\mathbf{w}$	Calibrated supervision weights	$\mathbf{w} \in \mathcal{W} \subset \mathbb{R}^k$
$\mathcal{W}$	Feasible set (probability simplex)	$\mathcal{W} = \{\mathbf{w} \mid w_i \geq \varepsilon,\ \sum_i w_i = 1\}$
ε	Weight floor constraint	$\varepsilon > 0$, typically 10^{-4}
α	Fidelity regularization weight	$\alpha \in [0.1, 2.0]$
β	Structure alignment weight	$\beta \in [0.1, 1.0]$
γ	Redundancy penalty weight	$\gamma \in [0.01, 0.5]$
δ	Bottleneck protection weight	$\delta \in [0.01, 0.2]$
$L(\mathbf{w})$	SCU objective function	Defined in Eq. (1)
$\mathbf{w}^*$	Optimal calibrated weight vector	$\mathbf{w}^* = \arg\min_{\mathbf{w}\in\mathcal{W}} L(\mathbf{w})$
$\nabla^2 L$	Hessian of SCU objective	Defined in Lemma 4.2
$\kappa(\mathbf{H})$	Condition number of Hessian	Bounded by Corollary 4.7
Evaluation Metrics		
ρ	Spearman rank correlation	$\rho \in [-1,1]$, Eq. (12)
τ	Kendall rank correlation coefficient	$\tau \in [-1,1]$
Operators and Conventions		
$\lVert\cdot\rVert_2$	Euclidean (ℓ_2) norm	$\lVert\mathbf{x}\rVert_2 = \sqrt{\sum_i x_i^2}$
$\lVert\cdot\rVert_1$	ℓ_1 norm	$\lVert\mathbf{x}\rVert_1 = \sum_i \lvert x_i\rvert$
$\odot$	Elementwise (Hadamard) product	$(\mathbf{a}\odot\mathbf{b})_i = a_i b_i$
$\succeq$	Positive semidefinite ordering	$\mathbf{R} \succeq 0$ means $\mathbf{R}$ is PSD
$\lambda_{\max}(\cdot)$	Largest eigenvalue	$\lambda_{\max}(\mathbf{R})$
$\lambda_{\min}(\cdot)$	Smallest eigenvalue	$\lambda_{\min}(\mathbf{R})$
$\mathbb{1}[\cdot]$	Indicator function	$\mathbb{1}[P] = 1$ if P is true, 0 otherwise
$\mathrm{Var}(\cdot)$	Variance of a vector	$\mathrm{Var}(\mathbf{w}) = \frac{1}{k}\sum_i (w_i - \bar{w})^2$

Table 2
Positioning of SC-FMA relative to prior work. Methods are classified along two axes: local vs. structural level of analysis (horizontal) and diagnostic vs. interventional function (vertical).

	Local / Individual Step	**Structural / Relational**
Diagnostic	Gradient Attr. [50, 51] ERASER [13], SHAP [32]	GNNExplainer [61] Network Robustness [2]
Interventional	PRMs [30, 56] Math-Shepherd [56]	**SC-FMA (this work)** Convex Calibration

Table 3
Data composition. Synthetic traces are used for method development, calibration, ablations, and error analysis. PRM800K provides real step-label ranking evidence under the frozen hash split. GSM8K/HotpotQA replay and downstream filtering routes remain failed or blocked provenance rather than positive validation evidence.

Dataset	**Train**	**Val**	**Test**	**Total**
Synthetic traces	120	40	40	200
Reflective steps	617	205	205	1,027
PRM800K locked samples	—	—	4,417	4,417
PRM800K labeled steps	—	—	34,219	34,219

Table 4
Hyperparameter search grid and optimal values. For each parameter, we report the search range, step size, optimal value, and the ΔSpearman ρ when varying the parameter by $\pm 20\%$ from the optimum (one-at-a-time sensitivity).

Param	**Range**	**Optimal**	$\Delta\rho$ $(+20\%)$	$\Delta\rho$ (-20%)
α (fidelity)	$[0.1, 2.0]$ step 0.1	1.0	−0.018	−0.031
β (structure)	$[0.1, 1.0]$ step 0.1	0.5	−0.007	−0.015
γ (redundancy)	$[0.01, 0.5]$ step 0.01	0.20	−0.004	−0.009
δ (bottleneck)	$[0.01, 0.2]$ step 0.01	0.10	−0.003	−0.006
ϵ (floor)	$\{10^{-4}, 10^{-3}, 10^{-2}\}$	10^{-4}	—	—
Ridge: α_c	$[0.5, 2.0]$ step 0.1	1.2	—	—
Ridge: α_n	$[0.1, 1.0]$ step 0.1	0.6	—	—
Ridge: τ (temp.)	$[0.5, 3.0]$ step 0.1	1.5	−0.011	−0.012

Table 5
Step importance ranking results. Mean metrics across 200 traces (1,027 steps). Bootstrap CIs at 95% (10k resamples). SC-FMA QP and Ridge are bolded. † indicates $p < 0.01$ vs. raw CIU (Wilcoxon signed-rank).

Method	**Spearman ρ**	**Kendall τ**	**NDCG@3**	**NDCG@5**
SC-FMA QP	**0.608**†	0.533	0.921	0.958
SC-FMA Ridge	0.533†	0.447	0.889	0.927
Gradient×Input	0.491	0.419	0.881	0.924
Raw CIU	0.483	0.412	0.874	0.919
Attention Rollout	0.461	0.388	0.863	0.912
SC-FMA Projection	0.338	0.278	0.806	0.872
MC Shapley (500 perm.)	0.524	0.451	0.895	0.933
Token Surprisal	0.310	0.252	0.791	0.863
Step Entropy	0.287	0.234	0.782	0.857
Span Length	−0.006	−0.013	0.740	0.826
Relative Position	−0.005	−0.014	0.740	0.825
Random	−0.010	−0.008	0.736	0.825
Oracle (ground truth)	1.000	1.000	1.000	1.000

Hyperparameter Sensitivity Analysis

The figure comprises four panels (a)–(d), each plotting Spearman ρ against one hyperparameter with the remaining three fixed at their optimal values:

- **(a)** $\rho(\alpha)$ – **Fidelity weight.** Concave profile peaking at $\alpha = 1.0$. At $\alpha < 0.5$, under-weighting CIU fidelity causes ρ to drop by -0.031; at $\alpha > 1.5$, over-weighting CIU relative to structure causes a smaller decline (-0.018), confirming that moderate structural regularization is beneficial.
- **(b)** $\rho(\beta)$ – **Structure alignment weight.** Broad optimum in $\beta \in [0.4, 0.6]$. Removing structure entirely ($\beta = 0$) costs -0.057 in ρ; overshooting ($\beta > 0.8$) costs -0.009 to -0.015, indicating that structure provides robust gains within a wide tolerance band.
- **(c)** $\rho(\gamma)$ – **Redundancy penalty weight.** Flat optimum around $\gamma = 0.20$. The sensitivity window is narrow (± 0.009 for $\pm 20\%$ variation), consistent with redundancy affecting only traces with non-trivial redundancy clusters (approximately 62% of the benchmark).
- **(d)** $\rho(\delta)$ – **Bottleneck protection weight.** Plateau between $\delta = 0.08$ and $\delta = 0.12$. Very flat ($\Delta\rho < 0.006$), reflecting that bottleneck nodes are sparse ($\approx 15\%$ of steps) and the log-barrier activates only at the boundary $w_i \to \varepsilon$.

All curves are averaged over 10 bootstrap resamples; shaded regions indicate ± 1 standard error. The dominance of α and β over γ and δ is consistent with the ablation hierarchy reported in Table 6.

Figure 1: Hyperparameter sensitivity curves. Panels (a)–(d) show the one-at-a-time effect of varying α, β, γ, and δ while holding other parameters fixed at their validation-set optima. The vertical dashed line in each panel marks the selected optimal value. Shaded regions indicate ± 1 standard error across 10 bootstrap resamples of the validation set.

Table 6
Extended ablation study. $\Delta\rho$ is the change in mean Spearman ρ from the full QP configuration ($\alpha = 1.0$, $\beta = 0.5$, $\gamma = 0.2$, $\delta = 0.1$, $\varepsilon = 10^{-4}$). p-values are from one-sided Wilcoxon tests against the full QP. Effect size $r = Z/\sqrt{N}$. Synergy ✓ indicates statistically significant positive interaction ($p < 0.05$).

Variant	α	β	γ	δ	ρ (Δ, p, r)
(0) SC-FMA QP (full)	1.0	0.5	0.2	0.1	**0.608** (—)
(1) − Structure	1.0	0.0	0.2	0.1	0.551 (−0.057, <0.001, 0.39)
(2) − Fidelity	0.1	0.5	0.2	0.1	0.536 (−0.072, <0.001, 0.43)
(3) − Redundancy	1.0	0.5	0.0	0.1	0.586 (−0.022, 0.003, 0.23)
(4) − Bottleneck	1.0	0.5	0.2	0.0	0.595 (−0.013, 0.008, 0.17)
(5) Structure only	0.0	0.5	0.0	0.0	0.438 (−0.170, <0.001, 0.56)
(6) Fidelity only (Raw CIU)	1.0	0.0	0.0	0.0	0.483 (−0.125, <0.001, 0.52)
(7) Fidelity + Structure	1.0	0.5	0.0	0.0	0.573 (−0.035, <0.001, 0.28)
(8) Fidelity + Redundancy	1.0	0.0	0.2	0.0	0.518 (−0.090, <0.001, 0.47)
(9) Fidelity + Bottleneck	1.0	0.0	0.0	0.1	0.504 (−0.104, <0.001, 0.50)

Structural Diagnostic: CIU versus Structural Necessity

The figure contains four subpanels that jointly characterize the relationship between local interventional utility (CIU) and graph-level structural necessity across the 200-trace synthetic benchmark:

- **(a) PRUNE mode.** Scatter plot of $\tilde{n}_i^{\text{prune}}$ vs. $\tilde{c}_i$ for all 1,027 reflective steps. Each point is one step; the diagonal reference line ($y = x$) marks perfect agreement. Pearson $r = 0.0753$, Spearman $\rho = 0.0596$. The vertical cluster at $\tilde{n}_i^{\text{prune}} = 0$ (67.79% of steps) dominates the distribution: many steps with $\tilde{c}_i \in [0.3, 0.8]$ have zero measured structural necessity.
- **(b) CASCADE mode.** Same layout for cascade-based necessity. Pearson $r = 0.0523$, $\rho = 0.0512$. The weaker correlation reflects that cascading removal (removing a node *and* its descendants) further decouples local utility from structural dependence—a cascaded node's local utility signal is independent of its downstream's structural role.
- **(c) BYPASS mode.** Bypass-based necessity (routing predecessors to successors). Pearson $r = 0.0917$, $\rho = 0.0623$, the highest among the three modes. This is expected: bypassing tests whether alternative paths exist, and high-CIU nodes are more likely to have alternative path support.
- **(d) Pooled density.** Kernel density estimate over all $(3 \times 1{,}027 = 3{,}081)$ mode-step pairs. The density is concentrated along the horizontal axis ($\tilde{n} \approx 0$) with a sparse upper tail, visually confirming the zero-inflation pattern (67.79% of all necessity values equal zero). The annotated region "Positive CIU, Zero Necessity" corresponds to 49.54% of steps—these are locally useful but structurally inert under graph ablation.

These diagnostics establish the empirical motivation for structural calibration: a naive weight vector $\mathbf{w} = \tilde{\mathbf{c}}$ would assign positive weight to nearly half of all steps that carry zero structural necessity, diluting supervision signals on the sparse bottleneck nodes that genuinely anchor the reasoning topology.

Figure 2: Structural diagnostic: local CIU versus graph-level structural necessity across three perturbation modes (PRUNE, CASCADE, BYPASS) plus a pooled density panel. The diagonal reference line marks perfect agreement ($y = x$). The zero-inflation annotation highlights that 67.79% of node-level necessity values are identically zero, and 49.54% of steps have positive CIU but zero structural necessity. These diagnostics motivate the SC-FMA calibration framework by exposing the systematic divergence between local utility signals and topological structural role. See Appendix B for additional structural diagnostic figures including mode comparison, redundancy distributions, compensation analysis, and resilience curves.

Table 7
Computational efficiency. Times are per-trace mean ± std over 40 test traces. Scaling column reports the exponent p in the power-law fit $T(k) \propto k^p$ estimated over traces with $k = 3, 5, 8, 12, 20$ (generated for scaling analysis).

Method	Wall-clock (ms)	Scaling $T \propto k^p$	Relative to QP
SC-FMA QP	6.8 ± 2.9	$p = 2.81$	1.0×
SC-FMA Ridge	0.4 ± 0.1	$p = 0.97$	0.06×
SC-FMA Projection	0.2 ± 0.05	$p = 0.98$	0.03×
Raw CIU (ablation)	0.8 ± 0.2	$p = 0.95$	0.12×
MC Shapley (500 perm.)	$3{,}420 \pm 890$	$p = 1.03$	503×
Gradient×Input	12.4 ± 3.1	$p = 1.12$	1.82×
Attention Rollout	8.1 ± 1.7	$p = 1.15$	1.19×
Token Surprisal	45.2 ± 8.3	$p = 1.08$	6.65×

Table 8
Time and space complexity comparison. Theoretical complexity is the asymptotic worst-case bound in big-$\mathcal{O}$ notation. Empirical scaling exponent p is the power-law fit $T(k) \propto k^p$. Space complexity accounts for all retained data structures (graph adjacency, redundancy matrix, Hessian).

Method	Time Complexity	Empirical p	Space Complexity	Convergence Guarantee
SC-FMA QP (SLSQP)	$\mathcal{O}(k^3)$	2.81	$\mathcal{O}(k^2)$	Superlinear (local)
SC-FMA Ridge	$\mathcal{O}(k)$	0.97	$\mathcal{O}(k)$	Closed-form
SC-FMA Projection	$\mathcal{O}(k)$	0.98	$\mathcal{O}(k)$	Closed-form
SC-FMA QP (warm-start)	$\mathcal{O}(k^3)$	2.81	$\mathcal{O}(k^2)$	Superlinear (local)
Raw CIU (masking)	$\mathcal{O}(k \cdot L)$	0.95	$\mathcal{O}(k \cdot L)$	Deterministic
MC Shapley (M perm.)	$\mathcal{O}(M \cdot k \cdot L)$	1.03	$\mathcal{O}(k \cdot L)$	$\mathcal{O}(1/\sqrt{M})$
Gradient×Input	$\mathcal{O}(k \cdot D)$	1.12	$\mathcal{O}(k \cdot D)$	Deterministic
Token Surprisal	$\mathcal{O}(k \cdot \ell \cdot D)$	1.08	$\mathcal{O}(k \cdot \ell \cdot D)$	Deterministic

Computational Scaling Analysis

The figure displays wall-clock time $T(k)$ as a function of trace size k on log-log axes, with power-law fits $T(k) \propto k^p$ for each method:

- **SC-FMA QP** ($p = 2.81$, solid blue). Follows the theoretical $\mathcal{O}(k^3)$ bound. For $k \leq 8$, $T(k) < 10$ ms; at $k = 20$, $T \approx 38$ ms, still sub-real-time for batch processing.
- **SC-FMA Ridge** ($p = 0.97$, dashed orange). Near-linear scaling with $T(k) < 0.5$ ms for $k \leq 8$, $T \approx 1.1$ ms at $k = 20$. Suitable for streaming and online inference.
- **SC-FMA Projection** ($p = 0.98$, dotted green). Slightly faster than Ridge (0.2 ms at $k = 5$), with identical asymptotic scaling.
- **MC Shapley (500 perm.)** ($p = 1.03$, dash-dot red). Linear in k but with a large constant factor ($\approx 3{,}400$ ms at $k = 5$), approximately 500× slower than SC-FMA QP.
- **Token Surprisal** ($p = 1.08$, dash-dot purple). Moderate scaling at 45 ms for $k = 5$, dominated by frozen LLM forward passes.

Error bars indicate ± 1 standard deviation over 10 traces per size point. All measurements are on an Intel Core i7-13700H CPU (32 GB RAM), single-threaded Python 3.11.

Figure 3: Computational scaling curves. Wall-clock time $T(k)$ as a function of trace size k (3–20 steps) on log-log axes. Each curve shows the power-law fit $T(k) \propto k^p$; the exponent p and corresponding $\mathcal{O}$ complexity class are annotated. SC-FMA QP follows the $\mathcal{O}(k^3)$ theoretical bound, while Ridge and Projection scale near-linearly. Error bars indicate ± 1 std. over 10 traces per size. Dashed lines show the power-law fits with $R^2 > 0.96$ for all methods.

Table 9
Current positive real-data evidence and claim boundaries. PRM800K supports real step-label ranking and an in-distribution frozen PRM baseline context only. GSM8K/HotpotQA replay and downstream filtering remain failed or blocked provenance, not positive validation evidence.

Route	Evidence type	Samples	Steps	Locked metric	Allowed interpretation
v3.6 PRM800K hash split	Real step-label ranking	4,417	34,219	`w_struct` Spearman 0.6113401179642559; raw local utility −0.07745914322519368	Supports `M_STEP_RANKING` and `M_STEP_RANKING_REAL_PRM800K`.
v3.8 frozen PRM locked scoring	In-distribution frozen PRM baseline context	4,417	34,219	Frozen PRM prefix-score Spearman 0.2515662235547571; `w_struct - prm` CI [0.34499208448462026, 0.37454675449147783]	Supports overlap-limited baseline context only; external generalization and PRM training claims are not allowed.
GSM8K/HotpotQA replay and filtering routes	Failed or blocked provenance	—	—	No passing locked validation artifact	Not current positive evidence; downstream PRM/filtering remains future validation.

Table B.10
Step importance ranking metrics stratified by trace size k. Mean Spearman ρ over traces of each size. Standard errors in parentheses (bootstrap, 10k resamples).

Method	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k \geq 7$
SC-FMA QP	0.582 (0.041)	0.601 (0.038)	0.615 (0.035)	0.622 (0.033)	0.634 (0.029)
SC-FMA Ridge	0.521 (0.044)	0.534 (0.040)	0.540 (0.037)	0.538 (0.035)	0.542 (0.032)
SC-FMA Proj.	0.328 (0.048)	0.341 (0.044)	0.345 (0.041)	0.339 (0.039)	0.342 (0.036)
Raw CIU	0.471 (0.045)	0.485 (0.041)	0.490 (0.038)	0.482 (0.036)	0.486 (0.033)
Δ(QP − CIU)	+0.111	+0.116	+0.125	+0.140	+0.148

Structural Diagnostic Mode Comparison

Bar charts comparing PRUNE, CASCADE, and BYPASS modes on three metrics: (i) Pearson r between CIU and structural necessity (left panel), (ii) Spearman ρ (center panel), and (iii) zero-necessity rate (right panel). All three modes show consistent weak alignment ($r < 0.10$, $\rho < 0.07$) and high zero-inflation ($\approx 68\%$). BYPASS exhibits marginally stronger correlation ($r = 0.0917$) than PRUNE ($r = 0.0753$) and CASCADE ($r = 0.0523$), reflecting that alternative-path availability correlates weakly with local utility.

Figure B.4: Structural diagnostic summary by perturbation mode. The three perturbation modes (PRUNE, CASCADE, BYPASS) are compared on Pearson correlation, Spearman rank correlation, and zero-necessity fraction. Error bars indicate 95% bootstrap confidence intervals. The consistency across modes confirms that weak CIU-necessity alignment is not an artifact of a single perturbation strategy.

Redundancy and Compensation Analysis

(a) Redundancy density histogram. Distribution of per-trace redundancy density $d_R = \frac{2}{k(k-1)} \sum_{i<j} R_{ij}$. The mode is at $d_R \approx 0.35$, indicating moderate pairwise substitutability among reflective steps. The right tail ($d_R > 0.6$) corresponds to traces with large redundancy clusters (4+ steps serving interchangeable verification roles). **(b) Compensation ratio distribution.** Per-node compensation ratios $C(v) = \sum_{u \in D(v)} \max(0, N_u^{\text{after}} - N_u^{\text{before}}) / \max(N_v^{\text{removed}}, \epsilon)$ under PRUNE mode. The distribution is concentrated at zero (88.4% of nodes show zero compensation), with a thin upper tail. Median compensation is 0.000 across all three modes, confirming that downstream redistribution of structural necessity after node removal is minimal.

Figure B.5: Redundancy density and compensation analysis. Panel (a): per-trace redundancy density distribution (mean 0.3842). Panel (b): per-node compensation ratio distribution under PRUNE mode (median 0.000, mean 0.0084). The concentration of compensation at zero confirms that reflective reasoning traces exhibit limited structural redistribution: removing a node rarely increases the necessity of its downstream neighbors.

Resilience under Ordered Node Removal

Four removal-order resilience curves plotting remaining structural necessity fraction against the fraction of nodes removed (x-axis: 0 to 1, y-axis: remaining necessity $N_{\text{rem}}/N_{\text{total}}$). **Necessity-first** (solid red): nodes are removed in descending order of structural necessity, producing the steepest initial decline (AUC = 0.1488). **Attribution-first** (dashed blue): nodes removed by descending CIU; the decline is much shallower (AUC = 0.4761), nearly indistinguishable from random. **Sequential** (dotted green): removes nodes in their original trace order (AUC = 0.4840). **Deterministic random** (dash-dot gray): uses a fixed hash-based ordering (AUC = 0.5098). The large gap between necessity-first and attribution-first curves (ΔAUC = 0.3273) confirms that CIU ranking does not approximate structural criticality ranking: removing high-CIU nodes degrades the graph structure similarly to random removal.

Figure B.6: Resilience curves under ordered node removal. Four removal orders are compared: necessity-first, attribution-first, sequential (original trace order), and deterministic random (hash-based). The y-axis shows remaining structural necessity fraction; the x-axis is the fraction of nodes removed. AUC values are annotated. The necessity-first curve degrades sharply (AUC 0.1488), while attribution-first (AUC 0.4761) is close to random (AUC 0.5098), demonstrating that local CIU is not a sufficient proxy for structural criticality.

Table B.11
Held-out Stage 2 results stratified by difficulty tier. Bootstrap 95% CIs from 10k resamples. "CI excludes 0" indicates whether the bootstrap interval is strictly positive.

Stratum	Size	Spearman ρ	95% CI	CI excludes 0
S_{high} (max divergence)	70	0.287	[0.108, 0.451]	Yes
S_{mid} (moderate)	70	0.142	[–0.043, 0.318]	No
S_{low} (min divergence)	70	0.091	[–0.092, 0.271]	No
S_{rand} (random audit)	70	0.113	[–0.061, 0.285]	No
Aggregate (all strata)	280	0.163	[0.092, 0.235]	Yes

Table B.12
Taxonomy-stratified ablation analysis. $\Delta\rho$ values are relative to the full QP configuration. Standard errors from bootstrap (10k resamples).

Category	Full ρ	Δ $(-\beta)$	Δ $(-\gamma)$	Δ $(-\delta)$	Count
ERROR_CORRECTION	0.642	−0.073	−0.018	−0.009	128
VERIFICATION	0.631	−0.068	−0.021	−0.011	144
PLANNING	0.598	−0.051	−0.031	−0.014	140
BACKTRACKING	0.587	−0.049	−0.035	−0.012	127
DECOMPOSITION	0.573	−0.055	−0.019	−0.015	126
CONSTRAINT_TRACKING	0.561	−0.052	−0.016	−0.008	140
RETRIEVAL	0.534	−0.047	−0.013	−0.009	134
UNCERTAINTY_MON.	0.529	−0.044	−0.011	−0.010	133

Table C.13
SLSQP convergence statistics by trace size k. Iteration counts and wall-clock times are mean ± std. over all traces of each size in the test set. Warm-start uses Ridge initialization; cold-start uses uniform $\mathbf{w}^{(0)} = (1/k, \ldots, 1/k)$.

k	Warm-start iters	Cold-start iters	Warm-start ms	Cold-start ms
3	2.8 ± 0.7	5.1 ± 1.3	2.1 ± 0.6	3.8 ± 0.9
4	3.9 ± 1.1	6.8 ± 1.7	3.8 ± 1.0	6.2 ± 1.5
5	5.2 ± 1.5	8.9 ± 2.2	5.9 ± 1.7	9.7 ± 2.3
6	6.5 ± 1.9	10.4 ± 2.8	8.2 ± 2.3	12.9 ± 3.4
7	7.8 ± 2.3	12.1 ± 3.1	10.8 ± 3.1	16.5 ± 4.1
8	9.1 ± 2.7	14.3 ± 3.5	13.7 ± 3.9	21.2 ± 5.0

Table C.14
Sensitivity of downstream metrics to graph construction parameters. Values are mean ± std. over the test set. Δ columns show the absolute change from the default configuration ($\tau_{\text{tfidf}} = 0.35$, $w_{\text{topical}} = 0.75$).

Configuration	Spearman ρ	$\Delta\rho$	Bottleneck count	Δ count
Default ($\tau = 0.35$, $w = 0.75$)	0.608	—	0.81	—
$\tau_{\text{tfidf}} = 0.28$ (−20%)	0.604	−0.004	0.78	−0.03
$\tau_{\text{tfidf}} = 0.42$ (+20%)	0.602	−0.006	0.85	+0.04
$w_{\text{topical}} = 0.60$ (−20%)	0.605	−0.003	0.79	−0.02
$w_{\text{topical}} = 0.90$ (+20%)	0.610	+0.002	0.83	+0.02